

LeakSense – Smart LPG Gas Leakage Detection and Automatic Safety System

CITS5506 The Internet of Things
The University of Western Australia, Australia
Semester 1, 2026

Group 27

1st Yuxuan Xi, 24319908@student.uwa.edu.au
2nd Yiming Zhao, 24139958@student.uwa.edu.au;
3rd Sahil Pankajbhai Patel, 24562775@student.uwa.edu.au;
4th Sohaib Ahmed Khan, 24756957@student.uwa.edu.au

Abstract

Residential LPG leakage is a serious household safety problem because accumulated gas can cause fire, explosion, and health hazards, yet affordable detectors typically provide only local alarms without physical mitigation or remote monitoring. This report presents LeakSense, a low-cost Internet of Things prototype that detects LPG risk, responds locally, and provides real-time cloud visibility. The system was developed using an iterative prototyping method and integrates an ESP32 microcontroller, a MEMS gas sensor, a BME280 atmospheric sensor, relay-driven actuators, Firebase Realtime Database, and a browser dashboard. The firmware samples gas voltage every two seconds and classifies readings into Safe, Warning, Danger, and Extreme states, escalating Warning to Danger when abnormal temperature rise or high absolute temperature indicates additional thermal risk. Local actuators operate independently of network connectivity, while Firebase uploads support live status, history logging, and browser notifications. Testing confirmed correct state classification across all thresholds, actuator response within the two-second polling interval, cloud-to-dashboard updates within three seconds, successful push notifications, and continued local safety operation during Wi-Fi interruption. The prototype cost approximately \$94 AUD. Key limitations are the prototype-scale fan, the absence of an automatic gas shutoff valve, and empirically determined voltage thresholds that require formal calibration against certified reference gas for production deployment.

Keywords

Internet of Things; LPG gas detection; ESP32; Firebase; dual-sensor escalation; residential safety; web dashboard

I. PROBLEM STATEMENT

LPG is used in over two million Australian households for cooking, heating, and hot water systems [1]. In its natural state, LPG is odourless and can accumulate to dangerous concentrations before occupants become aware of a leak. According to Fire and Rescue NSW, gas-related incidents account for a significant proportion of residential fire call-outs each year [2].

Existing residential gas detection solutions fail to address the full scope of this safety problem. Basic standalone detectors produce only a local audible alarm, which is ineffective when occupants are asleep, absent, or hearing-impaired, and do not initiate any physical hazard mitigation. Smart consumer detectors provide mobile push notifications but remain proprietary, do not control physical ventilation, and do not expose sensor data for custom analysis. Research-grade IoT prototypes have demonstrated remote alerting but typically use analog sensors without environmental compensation, rely on single-sensor detection, and lack multi-threshold response logic and cloud-connected dashboards [3]. Industrial detection systems provide certified shutoff valve integration and high measurement accuracy but are prohibitively expensive for residential installation.

A critical and unaddressed limitation across all affordable solutions is the absence of multi-factor escalation. Gas leakage in combination with a sudden rise in ambient temperature — which may indicate an ignition or fire risk — is not modelled in any existing consumer product. The central problem is therefore the absence of an affordable, integrated residential solution that combines: continuous gas voltage monitoring, graduated multi-threshold response with thermal escalation, automatic physical

hazard mitigation, and remote cloud-based monitoring with historical data logging. LeakSense is designed specifically to fill this gap.

II. INTRODUCTION

The Internet of Things enables physical sensor systems to communicate with cloud infrastructure and user-facing applications, creating safety systems that respond both locally and remotely. LeakSense applies this IoT architecture to the practically important problem of residential LPG gas leakage detection, extending prior single-sensor approaches with a dual-sensor escalation model that integrates gas concentration voltage with ambient temperature readings.

Prior work in this area falls into four broad categories. First, passive standalone detectors provide local alarms but no remote awareness or physical response. Second, smart consumer products add cloud notification but maintain proprietary data ecosystems that prevent custom integration. Third, academic IoT prototypes demonstrate cloud-connected gas detection using commodity microcontrollers but typically rely on single analog MQ-series sensors without environmental compensation [3] [4]. Fourth, industrial systems provide the most complete safety response but at a scale and cost entirely unsuitable for residential use. A consistent limitation across affordable solutions is the absence of automatic ventilation control, thermal escalation logic, and real-time web-based monitoring with historical trend data [12].

This report describes the design, implementation, and testing of LeakSense — a system that addresses the identified gaps using a FireBeetle ESP32-E microcontroller, a SEN0565 MEMS analog gas sensor, a BME280 atmospheric sensor, relay-controlled actuators, Firebase Realtime Database via HTTPS REST, and a plain HTML/CSS/JavaScript web dashboard. The system was designed using an iterative prototyping methodology. Success was defined as correct state classification across all thresholds, actuator response within two seconds, end-to-end cloud-to-dashboard latency within three seconds, and continued local safety operation during Wi-Fi interruption.

The remainder of this report is structured as follows. Section III describes the system architecture and hardware design. Section IV presents the firmware implementation including thermal escalation logic. Section V describes the web dashboard and cloud architecture. Section VI presents testing methodology and results. Section VII discusses limitations and future work. Section VIII presents the project cost. Section IX concludes the report. Section X provides instructions for hardware setup, firmware installation, Firebase configuration, and dashboard operation. The references follow the How-To Guide, and the appendices provide the code function explanations and source code listings.

III. SYSTEM ARCHITECTURE

A. Overview

LeakSense is structured around five IoT layers: sensing, computing, actuation, cloud communication, and user interface. The block diagram in Fig. 1 illustrates the five subsystems, their data flows, and their interconnections. Two distinct data paths exist: a local safety path from sensing through processing to actuation that operates continuously regardless of network availability, and a cloud path from processing through Firebase to the dashboard that operates on a best-effort basis.

The ESP32 is the central integration point. It polls the gas sensor and BME280 every two seconds, applies a two-pass state classification algorithm (voltage thresholds followed by a thermal escalation check), drives all local actuators independently of network availability, and publishes data to Firebase every two seconds for the live dashboard and every five minutes for historical records.

03a · SYSTEM ARCHITECTURE

CITS5506 · Group 27 · LeakSense

Dual-sensor escalation, local actuation, cloud monitoring.

The system collects gas and temperature data, evaluates risk locally on the ESP32, drives actuators for safety, and streams readings to the cloud for remote visibility.

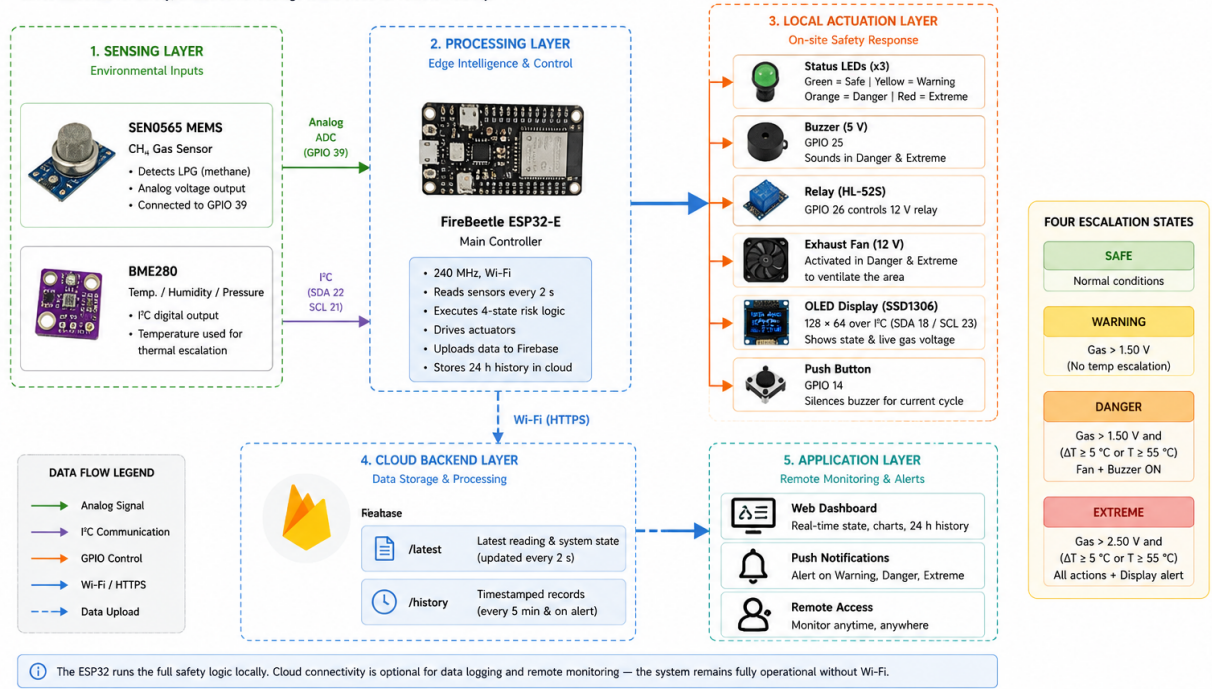


Fig. 1. LeakSense system block diagram showing all five IoT layers and their interconnections. (REST = Representational State Transfer; HTTPS = Hypertext Transfer Protocol Secure; LED = light-emitting diode; OLED = organic light-emitting diode.)

B. Component Overview

Table I lists all hardware components used in the LeakSense prototype.

TABLE I
LEAKSENSE HARDWARE COMPONENTS AND GPIO ASSIGNMENTS.

Component	Purpose	GPIO	Bus
FireBeetle ESP32-E	Central MCU — Wi-Fi, I2C, GPIO, 240 MHz dual-core	—	—
SEN0565 MEMS CH4	Analog gas voltage — primary input	39	ADC
BME280	Temp (escalation) + Humidity (display)	22/21	I2C-1
OLED SSD1306 128×64	Local real-time status display	18/23	I2C-0
HL-52S Relay	Controls exhaust fan	26	GPIO
12 V Exhaust Fan	Ventilation via relay	(relay)	—
5 V Buzzer	Audible alarm — 10 s cycle	25	GPIO
Green LED	Safe state indicator	17	GPIO
Yellow LED	Warning state indicator	16	GPIO
Red LED	Danger/Extreme indicator	4	GPIO
Push Button	Silences current buzzer cycle	14	GPIO

C. Component Descriptions

1) *FireBeetle ESP32-E*: The FireBeetle ESP32-E was selected because it integrates Wi-Fi, two I2C buses, ADC-capable GPIO, and a dual-core 240 MHz processor on one compact board. It is fully compatible with PlatformIO and the Arduino framework, which reduces firmware development time. The built-in Wi-Fi module enables direct HTTPS REST communication with Firebase Realtime Database without an additional networking component [9]. The FireBeetle ESP32-E board is shown in Fig. 2.

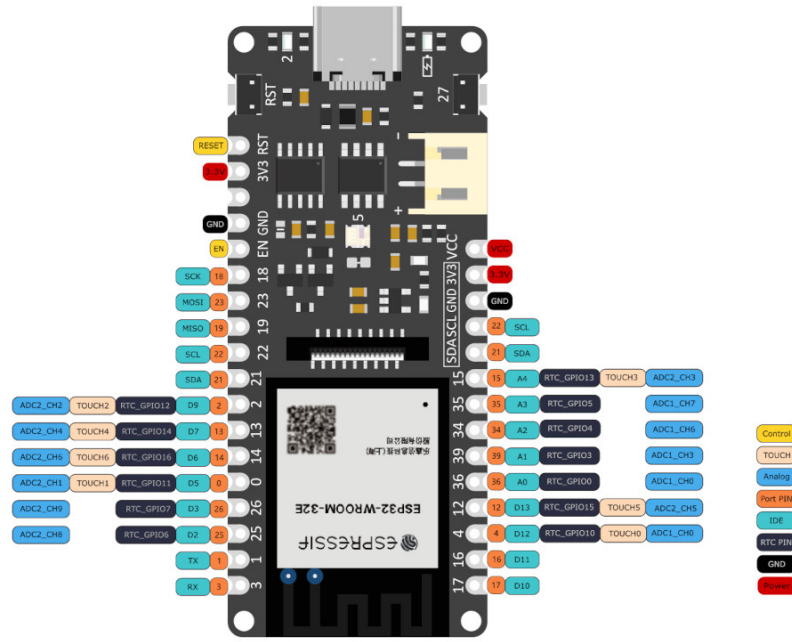


Fig. 2. FireBeetle ESP32-E Microcontroller serving as the central processing unit, responsible for sensor polling, state classification, actuator control, and cloud communication.

2) *SEN0565 MEMS CH4 Gas Sensor*: The SEN0565 Fermion MEMS CH4 sensor was selected as the primary gas detection input. The sensor outputs an analog voltage on GPIO 39 that rises with increasing gas concentration, without requiring manual R_s/R_0 resistance ratio conversion or sensitivity curve calculation. DFRobot documentation confirms the sensor is intended for qualitative measurement only and does not output calibrated concentration values in parts per million [5]. The sensor requires a warm-up period of at least five minutes before readings are stable, and 24 hours if it has been unused for an extended period [6].

Because no manufacturer-certified LPG-to-voltage mapping exists for this sensor, voltage thresholds were determined empirically by observing serial monitor output during controlled lighter-gas exposure in a laboratory environment. The resulting thresholds — 1.60 V for Warning and 1.90 V for Danger — are prototype calibration constants. For a production safety system, formal calibration against certified LPG reference gas or a known percentage lower-explosive-limit (LEL) standard would be required. The OSHA chemical data for LPG states that the LEL for propane is approximately 2.1% by volume and 1.9% for butane [7], which would serve as the reference concentration targets for proper calibration. The SEN0565 sensor module is shown in Fig. 3.



Fig. 3. SEN0565 MEMS CH4 Sensor providing an analog voltage output proportional to gas concentration, used as the primary input for state classification.

3) *BME280 Atmospheric Sensor*: The BME280 provides ambient temperature and humidity measurements over I2C on a dedicated bus (SDA GPIO 22, SCL GPIO 21) [10]. Temperature serves as a secondary escalation input: if the temperature rises 5 °C or more between consecutive two-second readings, or reaches 55 °C absolute, and the gas state is already Warning, the system escalates immediately to Danger and records `thermal_risk: true` in the Firebase payload. Humidity is displayed on the dashboard for environmental context and has no effect on the safety state. This dual-sensor approach is supported by Wu et al., who demonstrated that combining gas and temperature measurements significantly reduces false positive alarm rates compared to single-sensor designs [8]. The BME280 module is shown in Fig. 4.

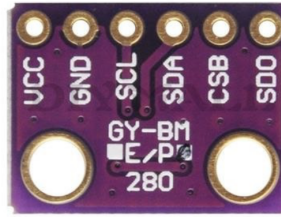


Fig. 4. BME280 Atmospheric Sensor providing temperature and humidity readings for thermal escalation and dashboard display.

4) *OLED Display SSD1306*: The SSD1306 0.96-inch I2C OLED display provides the local visual interface for the device. It was selected because it operates from 3.3 V logic and power, which is directly compatible with the ESP32 without level-shifting, and because its small footprint suits a prototype enclosure [13]. The display is connected to a dedicated I2C bus (TwoWire 0) on SDA GPIO 18 and SCL GPIO 23 at address 0x3C, separated from the BME280 bus to avoid address contention.

During operation the display shows the current temperature, humidity, gas voltage, baseline voltage, classified state (Safe, Warning, or Danger), and the on/off status of the fan and buzzer, as drawn by the `drawStatusScreen()` routine in Section IV. This makes the device usable as a standalone unit during demonstrations without opening the web dashboard, and it accelerated firmware debugging during development because sensor readings and state transitions could be observed directly on the device. The OLED module is shown in Fig. 5.



Fig. 5. 0.96-inch SSD1306 I2C OLED display providing local status output, showing live temperature, humidity, gas voltage, and classified safety state.

5) *Buzzer and LED Indicators*: The buzzer provides the audible local alarm when the system enters the Danger state. A Gravity digital buzzer module was selected because it is driven directly from a single GPIO line — it accepts a HIGH/LOW logic signal from the ESP32 and requires no PWM generation or external driver circuitry [14]. It is connected to GPIO 25 and operates on the ten-second on/off cycle described in Section IV.

Three through-hole LEDs provide additional visual status indication independent of the OLED: green (GPIO 17) for the Safe state, yellow (GPIO 16) for Warning, and red (GPIO 4) for Danger and Extreme. Each LED is wired through a $270\ \Omega$ current-limiting resistor to ground to keep forward current within the safe operating range for standard 5 mm LEDs at 3.3 V logic level. The three-colour scheme allows the system state to be identified at a glance from across a room, complementing the more detailed OLED readout. The buzzer module is shown in Fig. 6.



Fig. 6. Gravity digital buzzer module providing the audible alarm during Danger and Extreme states, driven from GPIO 25.

6) *Relay Module and Exhaust Fan:* The HL-52S 5 V single-channel relay module provides the switching interface between the ESP32 and the exhaust fan. The ESP32 cannot directly drive motor-class loads from its GPIO pins, so the relay isolates the microcontroller from the fan's switching transients while allowing a logic-level signal to control the higher-current path. The HL-52S is rated to switch up to 10 A at 28 VDC and draws approximately 60 mA at 5 V during activation, well within the prototype power budget [15]. It is driven from GPIO 26 with active-low polarity matching the module design.

The Delta 5 V DC frameless exhaust fan (40 mm, ~ 0.1 A, ~ 3150 RPM) is the project's physical mitigation device [16]. When the gas concentration crosses the Danger threshold, the relay closes the fan supply circuit and the fan disperses accumulated gas in the monitored space. This is the design choice that elevates LeakSense from a passive alarm system to an active safety system — the fan provides immediate hazard mitigation without waiting for occupant intervention. The 40 mm form factor is appropriate for prototype demonstration; a production deployment would substitute a mains-rated exhaust fan or integrate with existing kitchen ventilation, as discussed in Section VII. The fan is shown in Fig. 7.



Fig. 7. Delta 5 V DC 40mm frameless exhaust fan, switched via the HL-52S relay on GPIO 26 to disperse accumulated gas during Danger and Extreme states.

7) *Supporting Prototyping Components:* The remaining components support wiring, assembly, and stable testing of the prototype. An 830 tie-point solderless breadboard with dual isolated power rails was used as the prototyping platform because it provides sufficient connection density for the seven sensor and actuator modules without requiring soldered interconnects [17]. Male-to-male and male-to-female jumper wires route signals between the ESP32 and the sensor and output modules, with male-to-female ends used for the sensor and OLED headers that expose female pin sockets. A USB-A to USB-C cable supplies 5 V power to the FireBeetle ESP32-E and provides the serial programming and debugging interface to the development host. A push button on GPIO 14, configured with the ESP32 internal pull-up resistor, allows the operator to silence the current buzzer cycle as described in Section IV. A small project enclosure was used during testing to simulate a partially enclosed residential space in which leaked gas can accumulate to a detectable concentration around the SEN0565 sensing element.

D. Circuit Wiring

Two separate I2C buses are used. Bus 0 (TwoWire 0) operates on SDA GPIO 18 and SCL GPIO 23, serving the OLED SSD1306 display at I2C address 0x3C. Bus 1 (TwoWire 1) operates on SDA GPIO 22 and SCL GPIO 21, serving the BME280 at address 0x76 or 0x77. This separation prevents address conflicts. The SEN0565 analog output connects to GPIO 39, configured as a 12-bit ADC input with 11 dB attenuation for a full 0–3.3 V measurement range.

The relay module is driven from GPIO 26 (active-low, matching the HL-52S module polarity) and switches the exhaust fan's 12 V supply rail. The buzzer is driven from GPIO 25. Green, yellow, and red LEDs connect through 270 Ω current-limiting resistors to GPIO 17, GPIO 16, and GPIO 4 respectively. The push button connects GPIO 14 to ground with the ESP32 internal pull-up resistor enabled.

Fig. 8 shows the complete electrical setup used for the prototype. It identifies the ESP32 GPIO assignments, the two separate I2C buses, the SEN0565 analog input path, the LED indicator outputs, the alarm button input, and the relay-controlled fan and buzzer structure. This diagram therefore acts as the wiring reference for how the hardware subsystems are connected to the microcontroller.

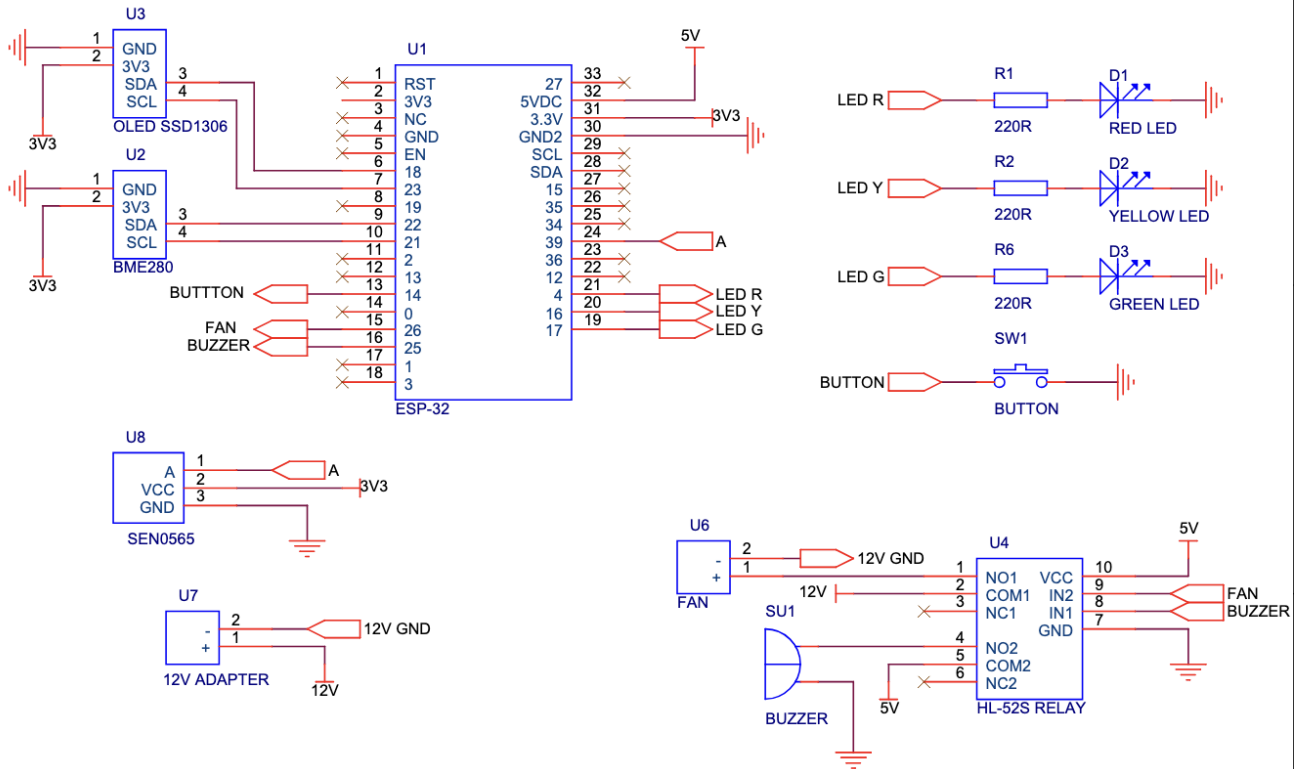


Fig. 8. LeakSense electronic schematic showing the ESP32 pin assignments, dual I2C sensor buses, SEN0565 analog input, LED indicators, push button, relay module, buzzer, fan, and 12V/5 V power connections.

Fig. 9 is the physical representation of the same circuit on the breadboard. It shows how the schematic connections were implemented in the prototype, including the ESP32-E placement, sensor modules, OLED display, relay module, fan, buzzer, LEDs, and jumper-wire routing used during testing.

LeakSense — Hardware Configuration and Wiring Diagram

CITS5506 IoT · Group 27 · Semester 1, 2026

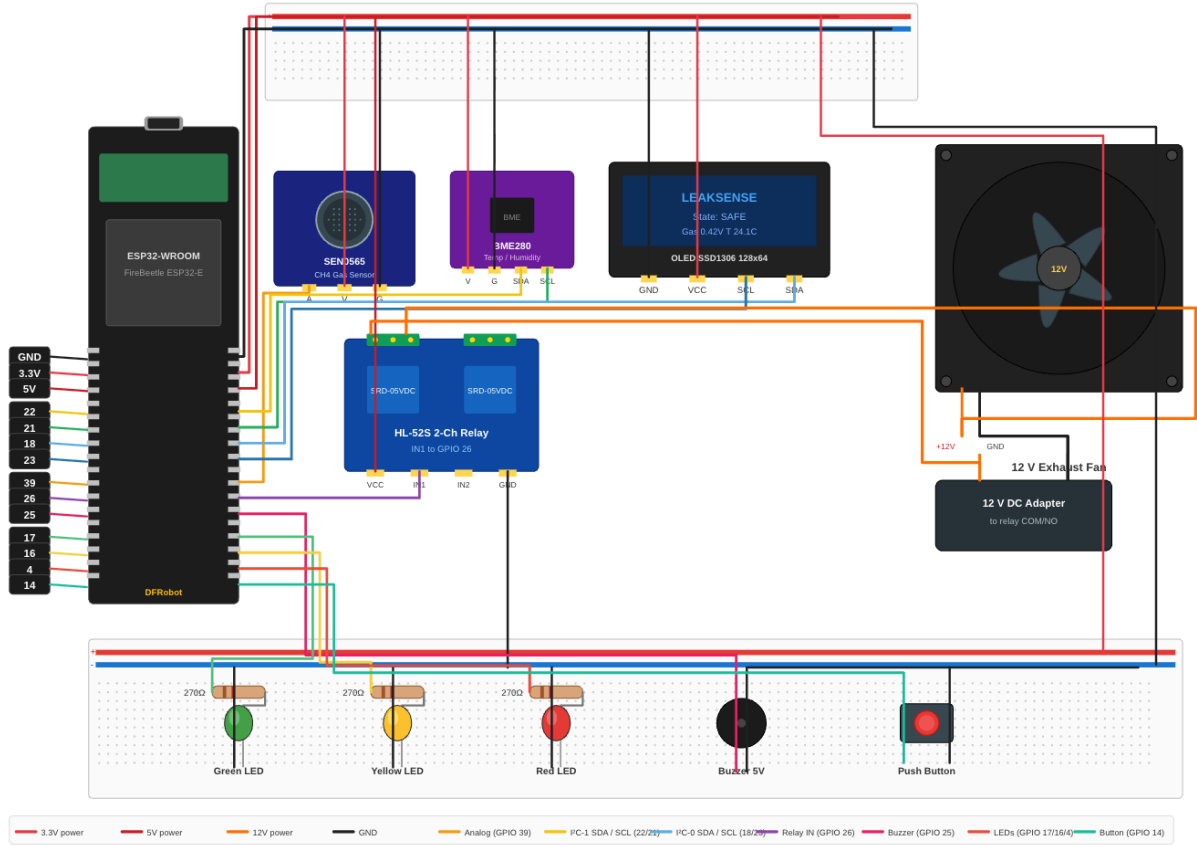


Fig. 9. LeakSense prototype breadboard circuit showing the ESP32-E, SEN0565, BME280, OLED, relay module, fan, buzzer, and LEDs.

IV. FIRMWARE IMPLEMENTATION

A. Design Approach

The firmware was developed in PlatformIO using the Arduino framework and an iterative prototyping methodology. Each subsystem — I2C sensors, state machine, thermal escalation, and Firebase REST publishing — was verified independently before the next dependency was added. This approach ensured working baselines at each stage and minimised debugging complexity during integration.

B. Boot Sequence

On power-up the firmware executes six initialisation steps:

- 1) All GPIO pins are set to safe defaults: green LED on, all alarm actuators off, relay off.
- 2) A relay self-test fires for 500 ms to confirm hardware continuity.
- 3) I2C bus 0 is initialised and scanned, followed by OLED display initialisation at address 0x3C.
- 4) I2C bus 1 is initialised and scanned, followed by BME280 initialisation at address 0x76/0x77.
- 5) The SEN0565 ADC input is configured on GPIO 39 with 12-bit resolution and 11 dB attenuation. A ten-second baseline period is enforced with a serial monitor warning to allow sensor pre-heating. The manufacturer recommends at least five minutes warm-up [6].
- 6) Wi-Fi connection is attempted with a ten-second timeout, followed by Firebase initialisation if connected. If Wi-Fi fails, the system continues in offline mode with all local safety functions active.

C. State Classification

State classification is a two-pass algorithm executed on every two-second polling cycle. The first pass derives the base state from the gas sensor voltage reading using fixed thresholds. The second pass applies the thermal escalation check. The firmware defines three hardware states; Table II defines the complete state-to-action mapping including the Extreme dashboard display label.

TABLE II
STATE MACHINE: VOLTAGE THRESHOLDS, FIRMWARE STATES, DASHBOARD LABELS, AND SYSTEM RESPONSES.

Voltage	Firmware	Dashboard	Hardware	Actions
< 1.60 V	SAFE	Safe	Green LED	Fan off · Buzzer off · Firebase updated every 2 s
1.60–1.89 V	WARNING	Warning	Yellow LED	Fan off · Buzzer off · History logged immediately
≥ 1.90 V	DANGER	Danger	Red LED	Fan ON via relay · Buzzer 10 s cycle · Firebase alert
≥ 2.20 V	DANGER*	Extreme*	Red LED	Identical to Danger. Extreme is a dashboard display label only.

**Note: Extreme is a dashboard-only display label applied when voltage exceeds 2.20 V. The firmware hardware response is identical to Danger — same red LED, same fan activation, same buzzer cycle. The Extreme label provides the remote user with an immediately recognisable indication that the reading is critically elevated. No additional firmware state or actuator behaviour is introduced.*

D. Thermal Escalation Logic

The thermal escalation check runs as the second pass of state classification, after the voltage-based base state has been determined. Two conditions can promote a Warning base state to Danger:

- The temperature reading has risen by 5 °C or more since the previous two-second polling cycle (indicating a sudden thermal event).
- The absolute temperature has reached or exceeded 55 °C (indicating sustained extreme heat).

If either condition is met and the base state is Warning, the final state is promoted to Danger and the Firebase payload includes the field `thermal_risk: true`. This allows the dashboard and alert log to distinguish a pure gas event from a combined gas-and-heat event, which may indicate ignition or fire risk. Temperature alone — with the gas voltage in the Safe range — never triggers the alarm.

A worked example from testing: gas voltage reads 1.70 V (Warning base state); temperature rises from 25.0 °C to 31.0 °C between readings (a 6.0 °C rise, exceeding the 5 °C threshold); thermal escalation fires; final state is Danger; Firebase receives `{"state": "DANGER", "thermal_risk": true}`.

E. Actuator Control

Actuator states are determined by the final classified state and applied on every polling cycle. In the Safe state, the green LED is active and all alarm actuators are deactivated. In Warning, the yellow LED is active; the fan and buzzer remain off, as Warning represents elevated concentration without immediate danger. In Danger or Extreme, the red LED is active, the relay drives the exhaust fan, and the buzzer starts a ten-second alarm cycle.

The buzzer operates on a ten-second on/off cycle managed by `millis()`-based timing. When the buzzer completes a ten-second active period, it checks the current state: if still Danger, a new cycle begins automatically. The push button on GPIO 14 silences the current cycle only — it sets a silence flag that suppresses the buzzer output until the next cycle boundary, at which point the silence flag resets and the buzzer restarts if the state remains Danger. A 50 ms debounce is applied to the button input. Actuator states are reapplied on every polling cycle rather than only on state transitions, ensuring that outputs can never become stuck in an incorrect state.

F. Cloud Communication

The ESP32 connects to the local Wi-Fi network using credentials stored in `firmware/bleeping/src/secrets.h`. Firebase communication uses direct HTTPS REST calls — no third-party Firebase library is used, eliminating dependency version conflicts on the ESP32 toolchain.

A `PUT` request is made to `/leaksense/latest.json` every two seconds, overwriting the existing document so the dashboard always shows the current reading [11]. A `POST` request is made to `/leaksense/history.json` every five minutes and immediately on any Warning or Danger event, appending a new record. Each payload includes: voltage, temperature, humidity, state string, fan boolean, buzzer boolean, `thermal_risk` boolean, and a server-side timestamp.

Cloud communication failure does not affect local actuator operation. Firebase calls are made after all actuator updates are applied, and any exception is silently caught. The firmware continues the polling loop regardless of HTTP response status.

G. Firmware Optimisation

Several design optimisations were applied to improve measurement reliability and reduce false activations.

ADC averaging. The SEN0565 analog output is sampled 32 times per reading cycle inside `readAnalogVoltage()`, and the mean value is used for state classification. This averaging suppresses ADC noise and sensor jitter that would otherwise cause spurious threshold crossings on individual samples.

Two-pass classification ordering. The state classification algorithm performs the voltage threshold check in the first pass and the BME280 thermal escalation check in the second pass. This ordering ensures that the computationally cheaper ADC read and voltage comparison execute before the I2C sensor read, keeping the critical path short when no escalation is required.

Gas level confirmation debounce. The `confirmGasLevel()` function requires five consecutive readings above a threshold before the state is promoted to Danger. This debounce prevents a single transient voltage spike — caused by sensor noise or a brief gas puff — from triggering full alarm activation, reducing nuisance alarms without increasing the worst-case detection latency beyond 10 seconds.

Adaptive baseline tracking. The baseline voltage follows ambient drift using a slow exponential moving average that updates only when the current reading is below the Warning threshold. This allows the system to adapt to gradual environmental changes such as temperature-induced sensor drift without incorporating leak events into the baseline, preserving threshold accuracy over time.

V. WEB DASHBOARD

A. Technology and Architecture

The web dashboard is a single-page application implemented in plain HTML, CSS, and JavaScript with no build tools or frameworks. Chart.js is loaded from CDN for the trend charts. The Firebase JavaScript SDK (compat version 10.12.0) is loaded from CDN for real-time database access. The dashboard is hosted on Firebase Hosting at `leaksense-iot.web.app`. A Chrome service worker enables browser push notifications via the Web Push API, delivering alerts even when the dashboard tab is not open.

The JavaScript is organised into five files: `secrets.js` provides the local Firebase web configuration from an ignored file; `mock.js` provides simulated sensor data for development without hardware; `charts.js` encapsulates all Chart.js initialisation and update logic; `dashboard.js` contains the main state management and DOM manipulation; `firebase.js` contains the Firebase initialisation and real-time listeners.

B. Dashboard Features

Table III summarises the dashboard features and their technical implementation.

TABLE III
LEAKSENSE WEB DASHBOARD FEATURES AND IMPLEMENTATION.

Feature	Description	Implementation
Live gas voltage	Current voltage with Safe/Warning/Danger/Extreme label	Firebase <code>onValue()</code> WebSocket listener
Status badge & banner	Colour-coded state indicator updating in real time	CSS class toggling via JS on every reading
Device states panel	Fan, buzzer, and all three LED states shown live	DOM updates from Firebase payload fields
60-reading live chart	Voltage over time with threshold lines at 1.60, 1.90, 2.20 V	Chart.js line chart updated on each reading
24-hour history chart	Gas, temperature, and humidity over 24 h — dual Y-axis	<code>leaksense/history</code> query, Chart.js
24-hour reading table	Tabular log: voltage, temp, humidity, state	History path rendered to DOM table
Alert log	Timestamped log of all Warning, Danger, Extreme events	JS array rendered to DOM on each state change
Push notifications	Browser notifications on alert — tab may be closed	Chrome service worker + Web Push API
Thermal risk flag	Alert log flags combined gas-and-heat events	<code>thermal_risk</code> field from Firebase payload

C. Real-Time Update Mechanism

The dashboard uses the Firebase `onValue()` listener which establishes a persistent WebSocket connection to the Realtime Database. When the ESP32 pushes a new reading to `leaksense/latest`, Firebase pushes the update to all connected clients immediately. The dashboard receives the update and calls the `onNewReading()` function, which updates all UI elements in sequence. The entire update cycle from new data arrival to UI render completes in under 100 ms in testing.

The Safe dashboard state, shown in Fig. 10, represents normal ambient operation. The live voltage is below the 1.60 V warning threshold, so the interface presents a green status banner, keeps the fan and buzzer marked as inactive, and confirms that only the green LED is enabled. This view is the default monitoring state for day-to-day use.

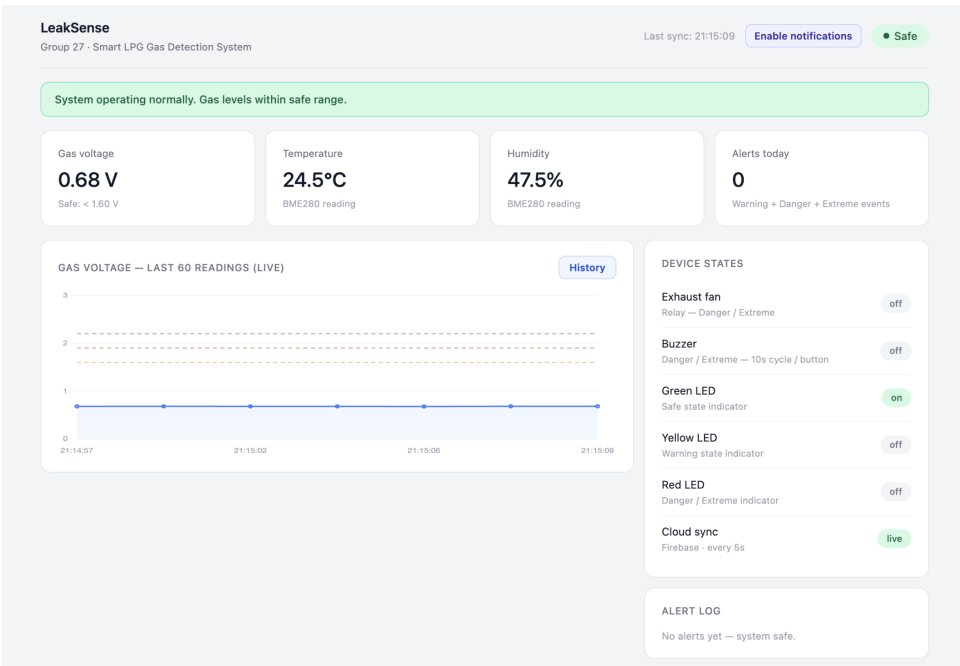


Fig. 10. LeakSense web dashboard — Safe state showing live gas voltage (0.68 V), system status, device states panel, and live trend chart.

When the gas voltage rises into the 1.60–1.89 V range, the dashboard changes to Warning as shown in Fig. 11. At this level, the system indicates elevated gas concentration but does not yet activate the fan or buzzer. The warning view is useful because it gives the user an early visual and remote indication before the system reaches the Danger threshold.

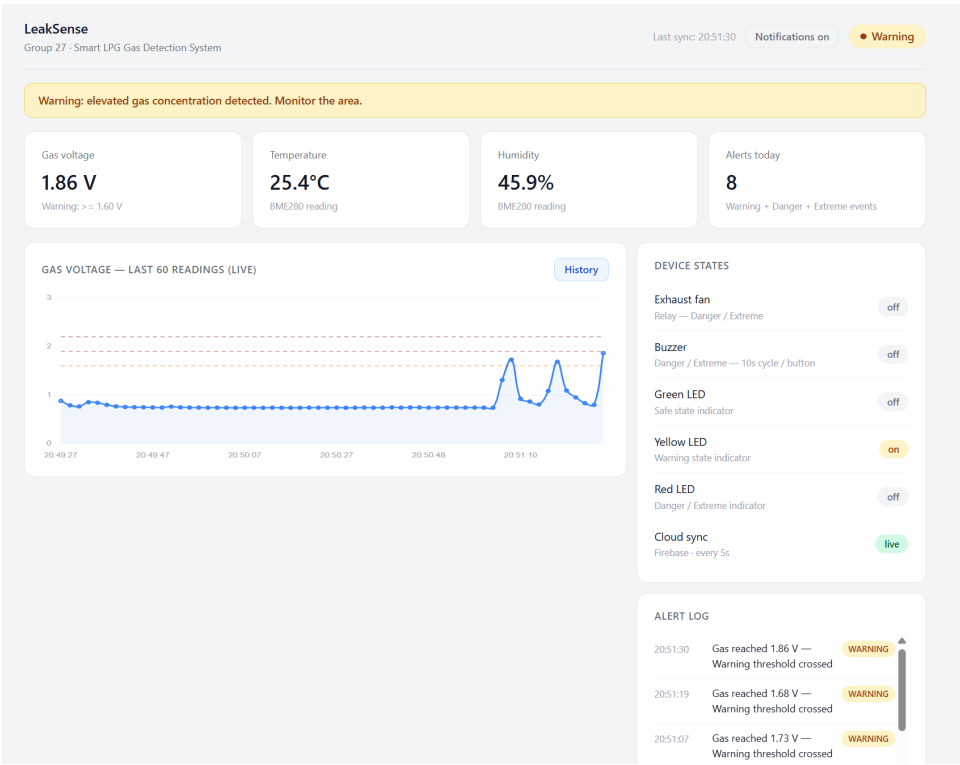


Fig. 11. LeakSense web dashboard — Warning state at 1.86 V with Chrome push notification visible.

Fig. 12 shows the Danger state, triggered when gas voltage reaches 1.90 V or above. In this state the dashboard confirms that the red LED is active, the relay-driven fan is on, and the buzzer alarm cycle has started. This matches the firmware behaviour

and verifies that the cloud dashboard is displaying the same safety state as the local hardware.

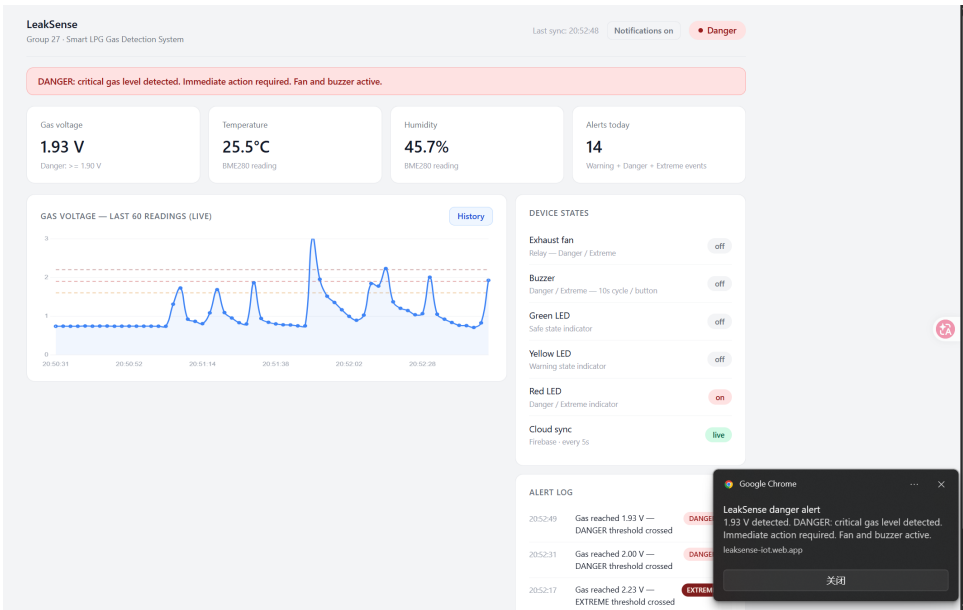


Fig. 12. LeakSense web dashboard — Danger state at 1.93 V with Chrome push notification visible.

The Extreme view in Fig. 13 is a dashboard-level escalation label for very high Danger readings above 2.20 V. The firmware still treats this condition as Danger, so the same local outputs remain active; however, the dashboard uses the Extreme label to make the remote alert more urgent and easier to distinguish from lower-level danger readings.

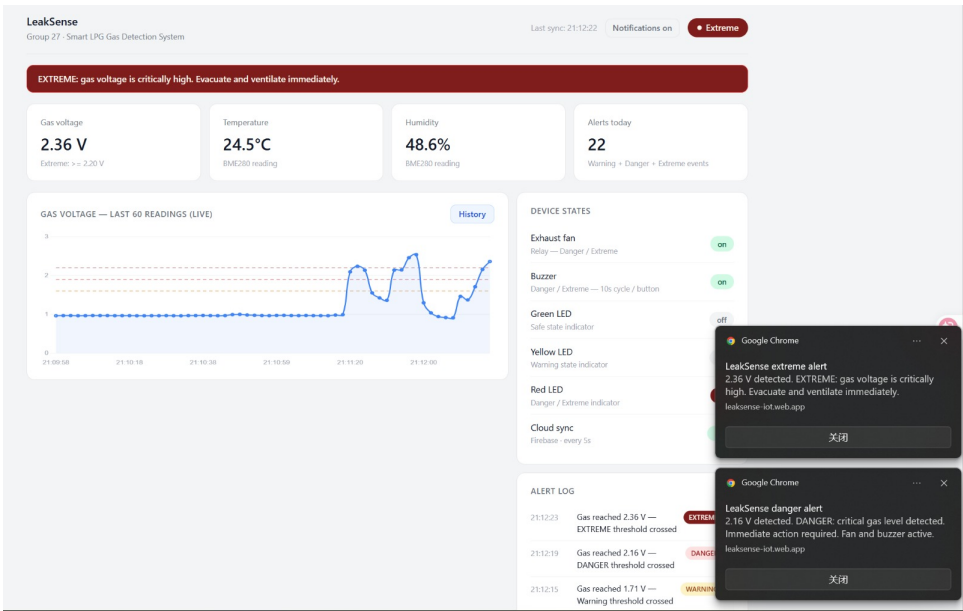


Fig. 13. LeakSense web dashboard — Extreme state at 2.36 V with Chrome push notification visible.

D. 24-Hour History View

The history view complements the live dashboard by showing how gas voltage, temperature, and humidity change over time. Rather than displaying only the latest Firebase value, it reads records from `leaksense/history` and plots the stored samples on a 24-hour chart. This helps identify recurring patterns, such as repeated short warning events, sensor warm-up drift, or environmental changes that coincide with gas readings.

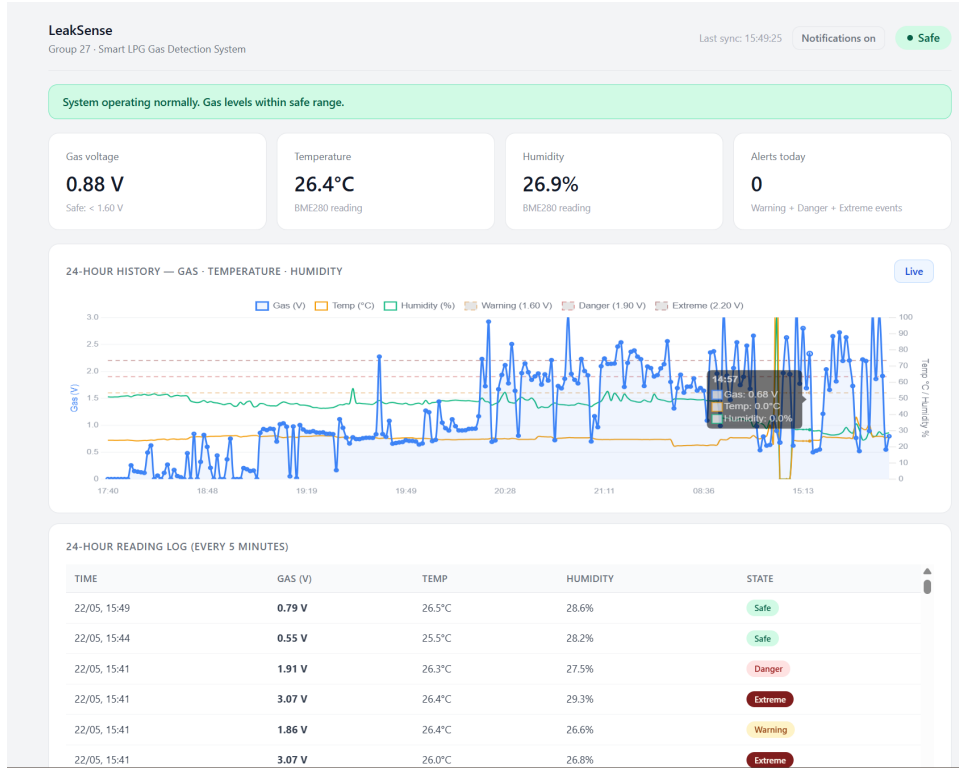


Fig. 14. LeakSense 24-hour history view showing stored sensor readings and state changes over time.

The table below the chart provides the same historical records in a readable log format, including timestamp, voltage, environmental values, and state. This is important for post-event analysis because a user can review when an alert occurred, how long elevated readings persisted, and whether the temperature trend supported the thermal escalation logic.

VI. TESTING AND RESULTS

A. Testing Methodology

Testing was conducted in four phases on the UWA IoT laboratory bench. The SEN0565 sensor was allowed to warm up for a minimum of five minutes before each test session, with the ten-second baseline period observed on every boot, consistent with manufacturer warm-up guidance [6].

Phase 1 verified individual component communication: I2C addresses were confirmed via serial monitor scan, sensor readings were checked against expected ambient values, and actuator outputs were verified with a multimeter and direct visual inspection.

Phase 2 tested state machine behaviour using butane lighter gas held at measured distances from the sensor, recording the voltages at which state transitions occurred. The thermal escalation logic was tested separately by observing the temperature delta field across consecutive readings during deliberate ambient temperature changes.

Phase 3 tested cloud communication by monitoring the Firebase console during sensor operation and measuring the time between a reading being taken and data appearing in the database.

Phase 4 tested the full end-to-end system including dashboard responsiveness, push notification delivery, 24-hour history logging, and behaviour during Wi-Fi disconnection.

B. Test Results

Table IV summarises the results of the nine key test cases conducted.

TABLE IV
LEAKSENSE TEST CASE RESULTS.

#	Condition	Expected	Observed	Time	Result
01	Ambient air (baseline)	Safe · green LED on	Safe · 0.68 V · green LED on	<1 s	PASS
02	Lighter gas — moderate	Warning · yellow LED	Warning at 1.86 V · yellow LED on	~2 s	PASS
03	Lighter gas — sustained	Danger · fan + buzzer	Danger at 1.93 V · fan + buzzer	~2 s	PASS
04	Lighter gas — direct (stove)	Danger fw · Extreme display	2.36 V · Extreme on dashboard	~2 s	PASS
05	Firebase latest push	Data within 2 s	Confirmed in console <2 s	<2 s	PASS
06	Dashboard live update	Update within 3 s	Dashboard updated <3 s	<3 s	PASS
07	Push notification	Chrome alert fires	Chrome notification received	~3 s	PASS
08	Wi-Fi disconnected mid-Danger	Local actuators continue	Fan + buzzer unaffected	0 s	PASS
09	Button press during alarm	Silences current cycle	Buzzer off · restarts next cycle	<0.1 s	PASS

C. Results Analysis

All nine test cases passed. The system correctly classified gas voltage into all four display states and activated the appropriate actuators within the two-second polling interval. The two-second response time is determined by the sensor polling interval rather than processing latency — classification and actuator update execute in under one millisecond once the sensor reading is available.

Firebase push latency of under two seconds is within the target and is primarily determined by network round-trip time. Dashboard update latency of under three seconds reflects the combined Firebase push latency and the `onValue()` callback overhead. Push notification delivery at approximately three seconds reflects additional service worker processing time.

The Wi-Fi disconnection test confirmed the most important safety property of the design: local actuators operate entirely independently of cloud connectivity. When Wi-Fi was disabled during a Danger state, the fan and buzzer continued operating and the OLED continued updating, while Firebase push attempts were silently skipped. The button test confirmed that the current buzzer cycle is silenced immediately on press but the subsequent cycle restarts automatically when the gas remains in the Danger state.

The live demonstration using a gas stove (no ignition) confirmed real-world operation: voltage climbed from a Safe baseline of 0.68 V to 2.36 V within four seconds of the stove valve opening, triggering Danger and the Extreme dashboard label, activating the fan and buzzer, and firing a Chrome push notification before any team member present could detect the gas by smell.

VII. LIMITATIONS AND FUTURE WORK

A. Current Limitations

1) *Empirical Voltage Thresholds*: The most significant limitation is that the voltage thresholds (1.60 V for Warning, 1.90 V for Danger) are prototype calibration constants determined empirically during laboratory testing. The SEN0565 MEMS sensor is documented by DFRobot as a qualitative sensor only — it does not produce calibrated concentration measurements [5]. No manufacturer-published voltage-to-concentration mapping exists for this sensor. For a production safety system, formal calibration against certified LPG reference gas or a known percentage-LEL standard (OSHA specifies propane LEL at 2.1% by volume [7]) would be required. The system architecture accommodates recalibration: threshold constants are defined at the top of the firmware source and can be updated without structural changes.

2) *Exhaust Fan Scale*: The exhaust fan used in this prototype is a small 12 V unit suitable for demonstrating relay-controlled ventilation but would not provide sufficient airflow to disperse LPG in a real room. A production system would require a mains-powered exhaust fan or a direct connection to an existing kitchen ventilation system.

3) *No Physical Gas Shutoff Valve*: The system activates ventilation and issues alerts but does not shut off the gas supply at the source. A solenoid valve integrated into the LPG supply line would provide a more complete safety response by eliminating the hazard rather than only mitigating it.

4) *Firebase Security Configuration*: The database currently operates with open read/write rules appropriate for a development prototype. A production deployment must implement Firebase Authentication and restrict write access to the ESP32 device credential only, with read access restricted to authenticated dashboard users.

5) *Single Sensor Node and No Battery Backup*: One gas sensor provides a single measurement point; a larger installation would benefit from multiple nodes. The system has no battery backup — a mains power failure disables all monitoring and local safety responses.

B. Future Work

Planned improvements include: (1) formal voltage-to-concentration calibration using certified LPG test gas to establish defensible thresholds based on percentage-LEL values; (2) a solenoid shutoff valve on the LPG supply line integrated with the existing relay driver circuit; (3) a UPS module or lithium battery backup for the ESP32; (4) production-grade Firebase security rules with device authentication; (5) a multi-node sensor mesh using ESP-NOW for larger residential spaces; (6) Firebase Cloud Functions for automatic history record pruning beyond 24 hours; and (7) dashboard email and SMS alert channels in addition to browser push notifications.

VIII. PROJECT COST

Table V gives the component cost estimate. Components were sourced from Core Electronics, Soldered, and Element14.

TABLE V
LEAKSENSE PROTOTYPE COMPONENT COSTS.

No.	Item	Description	Qty	Cost	Web Address
1	FireBeetle Board ESP32-E	Core controller with Wi-Fi and Bluetooth for IoT communication.	1	\$18.80	https://core-electronics.com.au/firebeetle-board-esp32-e-arduino-compatible.html
2	SEN0565 MEMS Methane CH4 Gas Detection Sensor	Primary gas sensing input used for leakage detection.	1	\$8.55	https://core-electronics.com.au/fermion-mems-methane-ch4-gas-detection-sensor-breakout-1-10000ppm.html
3	BME280 Atmospheric Sensor	Temperature, humidity, and pressure sensor used to reduce false alarms.	1	\$23.50	https://core-electronics.com.au/piicodev-atmospheric-sensor-bme280.html
4	OLED Display SSD1306 (I2C)	0.96 in I2C SSD1306 screen for local status and data monitoring.	1	\$7.20	https://core-electronics.com.au/white-i2c-oled-display-ssd1306.html
5	5 V Relay Module	Switches the fan load from ESP32 GPIO and provides electrical isolation.	1	\$3.95	https://core-electronics.com.au/5v-single-channel-relay-module-10a.html
6	Delta 5 V DC Exhaust Fan 40 mm	5 V DC 40 mm 3150 RPM fan used for ventilation simulation.	1	\$16.50	https://core-electronics.com.au/delta-electronics-frameless-fan-47192.html
7	Gravity Digital Buzzer Module	Gravity module used to provide audible alerts and system warnings.	1	\$3.14	https://core-electronics.com.au/digital-buzzer-module.html
8	5 mm LED	Visual state indicator.	3	\$3.14	https://au.element14.com/broadcom-limited/hlmp-3507/led-5mm-green-4-2mcd-569nm/dp/1003214
9	Resistors (270 Ohm)	Current limiting for LEDs; 600-pack covers all values.	1	\$2.95	https://core-electronics.com.au/600-pack-of-1-4-watt-1-resistors-30-values-20-of-each.html
10	Breadboard 830 tie-pt	Prototyping platform for all connections.	1	\$2.95	https://core-electronics.com.au/solderless-breadboard-830-tie-point-zy-102.html
11	Jumper Wires M-M and M-F	Breadboard interconnects; M-F wires are used for sensor modules and the OLED.	1	\$3.59	https://core-electronics.com.au/solderless-breadboard-jumper-cable-wires-75-pieces.html
Total				\$94.27	

Competing commercial products provide context for this cost. The Aqara Smart Gas Detector (AUD \$130) detects methane, not LPG. The Google Nest Protect (AUD \$189) detects smoke and carbon monoxide, not LPG. The Honeywell BW Solo detects LPG but has no auto ventilation and costs approximately AUD \$700. The Flowtech safety system includes a shutoff valve but starts at AUD \$1,500 and is designed for commercial installations. LeakSense delivers LPG detection, relay-driven fan ventilation, cloud monitoring, 24-hour history, and browser push notifications for approximately AUD \$94. The thermal escalation logic — gas plus temperature risk combined — is absent from every commercial alternative listed above.

IX. CONCLUSION

LeakSense demonstrates that an affordable, integrated residential LPG safety system can be built from commodity IoT components for approximately AUD \$94 — roughly one-seventh the cost of the nearest commercial product with comparable functionality, and one-sixteenth the cost of systems offering certified shutoff integration. The prototype integrates a MEMS analog gas sensor, a BME280 atmospheric sensor, relay-driven actuators, an ESP32 microcontroller, Firebase Realtime Database over HTTPS REST, and a browser-accessible dashboard with 24-hour history and push notifications into a coherent five-layer IoT architecture.

The four design goals defined in Section II were achieved and verified through nine test cases. Voltage-based state classification correctly identified Safe, Warning, Danger, and Extreme conditions at every tested threshold. The thermal escalation

logic correctly promoted Warning to Danger when a temperature rise of 5 °C or higher coincided with elevated gas readings, supporting the dual-sensor approach as a practical mechanism for distinguishing pure gas events from combined gas-and-heat events. Actuators responded within the two-second polling interval, cloud-to-dashboard latency remained under three seconds, and the Wi-Fi disconnection test confirmed the project’s most important safety property: all local actuation continues independently of cloud connectivity.

Beyond the working prototype, the project contributes a reproducible reference architecture for low-cost residential safety IoT. The separation of time-critical safety logic on the microcontroller from non-critical telemetry and visualisation in the cloud is a pattern that generalises to other household hazard domains, including smoke, carbon monoxide, and water leak detection. The empirical voltage thresholds, prototype-scale fan, and absence of a physical shutoff valve are acknowledged limitations rather than design flaws; each has a defined remediation path in Section VII, and none require structural redesign of the existing architecture.

For practitioners, the principal takeaway is that the technical gap between affordable consumer detectors and certified industrial systems is smaller than current market pricing implies. With formal calibration against a certified LPG reference, a mains-rated exhaust fan, a solenoid shutoff valve, and production-grade Firebase security rules, the LeakSense architecture provides a credible foundation for a deployable open residential gas safety product.

X. HOW-TO GUIDE

This guide describes how to assemble, configure, and run the LeakSense prototype from scratch.

A. Hardware Setup

- 1) Connect the SEN0565 gas sensor analog output to GPIO 39. Connect VCC to 3.3 V and GND to GND.
- 2) Connect the OLED SSD1306 to I2C Bus 0: SDA to GPIO 18, SCL to GPIO 23. Connect VCC to 3.3 V.
- 3) Connect the BME280 to I2C Bus 1: SDA to GPIO 22, SCL to GPIO 21. Connect VCC to 3.3 V.
- 4) Connect the HL-52S relay IN1 signal pin to GPIO 26. Connect the relay NO terminal in series with the fan’s positive supply.
- 5) Connect the buzzer signal pin to GPIO 25.
- 6) Connect the green LED anode through a 270 Ω resistor to GPIO 17, cathode to GND.
- 7) Connect the yellow LED anode through a 270 Ω resistor to GPIO 16, cathode to GND.
- 8) Connect the red LED anode through a 270 Ω resistor to GPIO 4, cathode to GND.
- 9) Connect one terminal of the push button to GPIO 14, the other to GND. The firmware enables the internal pull-up resistor.
- 10) Power the ESP32 via USB-C. Verify I2C connections using the serial monitor I2C scan output before uploading firmware.

B. Firmware Installation

- 11) Install VS Code and the PlatformIO IDE extension.
- 12) Open the `firmware/` folder as a PlatformIO project.
- 13) Copy `firmware/bleeping/src/secrets.h.example` to `firmware/bleeping/src/secrets.h` and fill in: `WIFI_SSID`, `WIFI_PASSWORD`, `FIREBASE_HOST`, and `FIREBASE_AUTH`.
- 14) Ensure the following are listed in `platformio.ini` under `lib_deps`: Adafruit GFX Library, Adafruit SSD1306, Adafruit BME280 Library, and mobitz/Firebase ESP32 Client.
- 15) Click the PlatformIO Upload button to compile and flash the firmware.
- 16) Open the Serial Monitor at 115200 baud. Allow five minutes for sensor warm-up before trusting readings.

C. Firebase Setup

- 17) Go to firebase.google.com and create a project named `leaksense-iot`.
- 18) Enable Realtime Database in test mode.
- 19) In Project Settings → Service Accounts → Database Secrets, copy the database secret into the local firmware `secrets.h` file as `FIREBASE_AUTH`.
- 20) Copy the databaseURL from Project Settings and paste it (without `https://`) into the same `secrets.h` file as `FIREBASE_HOST`.
- 21) Set database rules to allow read and write for development. Update rules before any production deployment.

D. Dashboard Setup

- 22) Copy `dashboard/js/secrets.example.js` to `dashboard/js/secrets.js`. The local `secrets.js` file is ignored by Git and should not be committed.
- 23) Paste the Firebase web config object from Project Settings → General → Your apps into `dashboard/js/secrets.js`.

- 24) Open `dashboard/index.html` in a browser. The dashboard connects to Firebase automatically and displays live data.
- 25) To enable push notifications, open the dashboard in Chrome and click Enable Notifications. Accept the browser permission prompt. Notifications will fire for Warning, Danger, and Extreme events.
- 26) The 24-hour history view is accessible by clicking the History button on the live chart.

REFERENCES

- [1] Frontier Economics, “Gas Vision 2050: Delivering a Clean Energy Future,” Gas Energy Australia, Sep. 2020. [Online]. Available: <https://www.gasenergyaus.au/get/1868/gas-vision-2050-frontier-economics-september.pdf>. [Accessed: May 2026].
- [2] Fire and Rescue NSW, “Annual Report 2024–25,” Fire and Rescue NSW, Oct. 2025. [Online]. Available: https://www.fire.nsw.gov.au/_data/assets/pdf_file/0022/4936/annual_report_2024_25.pdf. [Accessed: May 2026].
- [3] M. R. Islam *et al.*, “IoT-based smart gas leakage detection and safety system using wireless sensor networks,” *IEEE Access*, vol. 8, pp. 201360–201371, 2020, doi: 10.1109/ACCESS.2020.3035486.
- [4] R. Sarkar *et al.*, “Elevating Gas Safety Standards: ESP32-based Detection Systems with Blynk Integration,” in *Proc. 2024 IEEE Region 10 Symposium (TENSYP)*, 2024.
- [5] DFRobot, “SEN0565 Fermion MEMS CH4 Gas Detection Sensor Wiki,” DFRobot, 2024. [Online]. Available: <https://wiki.dfrobot.com/sen0565>. [Accessed: May 2026].
- [6] DFRobot, “SEN0565 Raw Reading Example — warm-up guidance,” DFRobot, 2024. [Online]. Available: <https://wiki.dfrobot.com/sen0565/docs/18020>. [Accessed: May 2026].
- [7] OSHA, “LPG Chemical Data — Lower Explosive Limit,” United States Department of Labor, 2024. [Online]. Available: <https://www.osha.gov/chemicaldata/484>. [Accessed: May 2026].
- [8] L. Wu, L. Chen, and X. Hao, “Multi-sensor data fusion algorithm for early indoor fire warning based on BP neural network,” *Information*, vol. 12, no. 2, p. 59, 2021.
- [9] Espressif Systems, “ESP32 Technical Reference Manual,” Espressif Systems, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. [Accessed: May 2026].
- [10] Bosch Sensortec, “BME280 Combined Humidity and Pressure Sensor — Data Sheet,” Document BST-BME280-DS002, Rev. 1.6, Sep. 2018. [Online]. Available: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf>. [Accessed: May 2026].
- [11] Google Firebase, “Firebase Realtime Database Documentation,” Google, 2024. [Online]. Available: <https://firebase.google.com/docs/database>. [Accessed: May 2026].
- [12] F. Saeed, A. Paul, A. Rehman, W. H. Hong, and H. Seo, “IoT-based intelligent modeling of smart home environment for fire prevention and safety,” *J. Sensor Actuator Networks*, vol. 7, no. 1, p. 11, Mar. 2018, doi: 10.3390/jsan7010011. [Online]. Available: <https://www.mdpi.com/2224-2708/7/1/11>. [Accessed: May 2026].
- [13] Core Electronics, “White I2C OLED Display (SSD1306),” Core Electronics, 2020. [Online]. Available: <https://core-electronics.com.au/white-i2c-oled-display-ssd1306.html>. [Accessed: May 2026].
- [14] DFRobot, “Gravity: Digital Buzzer Module,” DFRobot Wiki, 2024. [Online]. Available: <https://wiki.dfrobot.com/dfr0032/>. [Accessed: May 2026].
- [15] Core Electronics, “5V Single Channel Relay Module 10A,” Core Electronics, 2024. [Online]. Available: <https://core-electronics.com.au/5v-single-channel-relay-module-10a.html>. [Accessed: May 2026].
- [16] Core Electronics, “Delta Electronics Frameless Fan,” Core Electronics, 2024. [Online]. Available: <https://core-electronics.com.au/delta-electronics-frameless-fan-47192.html>. [Accessed: May 2026].
- [17] Core Electronics, “Solderless Breadboard 830 Tie-Point ZY-102,” Core Electronics, 2024. [Online]. Available: <https://core-electronics.com.au/solderless-breadboard-830-tie-point-zy-102.html>. [Accessed: May 2026].

APPENDIX A CODE FUNCTION EXPLANATIONS

The code function explanations and source listings document the implementation used by the LeakSense prototype. The same source code is also available in the project GitHub repository: <https://github.com/sahilppatel1807/CITS5506-LeakSense-IoT>. For hardware assembly, firmware upload, Firebase setup, and dashboard operation instructions, see the How-To Guide.

A. Code Organisation

The LeakSense implementation is divided into embedded firmware and browser dashboard code. The embedded firmware is stored in `firmware/` and runs on the FireBeetle ESP32-E using PlatformIO and the Arduino framework. The dashboard is stored in `dashboard/` and runs as a plain HTML, CSS, and JavaScript application connected to Firebase Realtime Database. This split keeps time-critical safety logic on the ESP32 and presentation logic in the browser.

Table VI summarises the files included in Appendix B. The source listings are included directly from the repository so the report shows the same code used by the prototype.

TABLE VI
LEAKSENSE SOURCE FILES INCLUDED IN APPENDIX B.

File	Role
firmware/blinking/src/main.cpp	ESP32 firmware: sensors, state machine, actuators, OLED display, Wi-Fi, Firebase REST upload.
firmware/blinking/platformio.ini	PlatformIO board, framework, monitor, and library dependency configuration.
firmware/blinking/src/secrets.h.example	Template for private Wi-Fi and Firebase credentials.
dashboard/index.html	Dashboard page structure and UI elements.
dashboard/css/style.css	Dashboard layout, state colours, cards, buttons, tables, and responsive styling.
dashboard/js/secrets.example.js	Template for the local Firebase web configuration file used by the dashboard.
dashboard/js/firebase.js	Firebase setup, database listeners, and data normalisation.
dashboard/js/dashboard.js	Main dashboard state handling, alert log, notifications, and DOM updates.
dashboard/js/charts.js	Chart.js live trend and 24-hour history charts.
dashboard/js/mock.js	Simulated readings for testing the dashboard without live hardware.
dashboard/service-worker.js	Browser notification service worker.

B. Firmware Functions Explained

The firmware is written with descriptive constants and small functions so that each hardware responsibility is visible in the code. Pin numbers, timing intervals, thresholds, active-low polarity, and calibration values are declared at the top of `main.cpp`. This makes the safety thresholds and GPIO assignments easy to audit and change.

The most important firmware functions are:

- `setup()` initialises GPIO, runs the LED self-test, configures ADC resolution, scans both I2C buses, initialises the OLED and BME280, and starts Wi-Fi connection in the background.
- `loop()` is the main cooperative scheduler. It services alarm controls and Wi-Fi continuously, then performs a complete sensor, classification, actuator, Firebase, and OLED update every two seconds.
- `readAnalogVoltage()` averages 32 ADC samples from the SEN0565 gas sensor, reducing noise before state classification.
- `updateGasBaseline()` tracks the clean-air baseline slowly when readings remain stable, but avoids learning elevated gas readings as normal.
- `evaluateGasLevel()` converts gas voltage into Safe, Warning, or Danger using the fixed thresholds.
- `confirmGasLevel()` prevents noisy single-sample danger spikes from triggering ordinary Danger, while allowing immediate Danger when the reading exceeds the Extreme voltage threshold.
- `hasThermalRisk()` checks whether the BME280 temperature has risen by at least 5 °C between readings or reached 55 °C absolute.
- `startDangerAlarm()`, `updateDangerAlarm()`, and `stopDangerAlarm()` control the buzzer and fan alarm cycle.
- `handleAlarmCancelButton()` handles the GPIO 14 push button and pauses detection temporarily after user acknowledgement.
- `firebasePayload()`, `uploadLatestToFirebase()`, and `uploadHistoryToFirebase()` build JSON records and send them to Firebase using HTTPS REST calls.
- `drawStatusScreen()` renders local OLED status including temperature, humidity, gas voltage, baseline, state, alarm, and fan status.

The firmware comments explain why timing, baselining, and upload behaviour are implemented in this way. The most important safety choice is that actuator updates occur before Firebase communication, so the fan, buzzer, and LEDs do not depend on Wi-Fi or cloud availability.

C. Dashboard Functions Explained

The dashboard JavaScript separates Firebase input, UI state handling, charts, and mock data. This makes it possible to test the browser interface without the ESP32 while keeping the same internal reading format.

The key dashboard functions are:

- `secrets.example.js` shows the required Firebase web configuration fields. The real local file is named `secrets.js` and is ignored by Git so project details are not committed.

- `normaliseReading()` in `firebase.js` converts Firebase records into one consistent object shape, handling missing fields, legacy names, booleans, numbers, timestamps, and state classification.
- `listenToSensorData()` subscribes to `leaksense/latest` so the dashboard updates in real time when the ESP32 uploads a new reading.
- `loadHistory()` and `listenToHistory()` read and stream `leaksense/history` records for the 24-hour history view.
- `onNewReading()` in `dashboard.js` is the main UI update function. It updates the state badge, banner, metrics, device state pills, live chart, alert log, and notifications.
- `sendAlertNotification()` creates browser notifications for Warning, Danger, and Extreme events.
- `setupDashboardViews()` switches between the live dashboard and 24-hour history views.
- `initChart()`, `updateChart()`, `initHistoryChart()`, and `addHistoryPoint()` in `charts.js` create and update the Chart.js visualisations.
- `getMockReading()` in `mock.js` produces simulated gas, temperature, humidity, actuator, and state values when hardware data is unavailable.

The HTML file defines the visible dashboard structure, the CSS file defines the state-specific visual design, and the JavaScript files apply the live Firebase data to those elements.

APPENDIX B SOURCE CODE LISTINGS

A. Firmware Source Code

Listing 1. ESP32 firmware source code: `firmware/blinking/src/main.cpp`.

```

1  #include <Arduino.h>
2  #include <Wire.h>
3  #include <WiFi.h>
4  #include <HTTPClient.h>
5  #include <WiFiClientSecure.h>
6  #include <Adafruit_GFX.h>
7  #include <Adafruit_SSD1306.h>
8  #include <Adafruit_BME280.h>
9  #include "secrets.h"
10
11 constexpr uint8_t kLedPin = D9;
12 constexpr uint8_t kGreenLedPin = 17;
13 constexpr uint8_t kRedLedPin = 4;
14 constexpr uint8_t kYellowLedPin = 16;
15 constexpr uint8_t kAlarmCancelButtonPin = 14;
16 constexpr uint8_t kOledSdaPin = 18;
17 constexpr uint8_t kOledSclPin = 23;
18 constexpr uint8_t kBmeSdaPin = 22;
19 constexpr uint8_t kBmeSclPin = 21;
20 constexpr uint8_t kAlarmOutputPin = 25;
21 constexpr uint8_t kFanRelayPin = 26;
22 constexpr int kSen0565AnalogPin = 39;
23 constexpr uint8_t kScreenWidth = 128;
24 constexpr uint8_t kScreenHeight = 64;
25 constexpr uint8_t kOledAddresses[] = {0x3C, 0x3D};
26 constexpr uint8_t kBmeAddresses[] = {0x76, 0x77};
27 constexpr uint8_t kGasSamplesPerRead = 32;
28 constexpr unsigned long kBlinkIntervalMs = 500;
29 constexpr unsigned long kRedBlinkIntervalMs = 75;
30 constexpr unsigned long kSensorReadIntervalMs = 2000;
31 constexpr unsigned long kFirebaseHistoryIntervalMs = 300000;
32 constexpr unsigned long kWifiConnectTimeoutMs = 15000;
33 constexpr unsigned long kWifiRetryIntervalMs = 30000;
34 constexpr unsigned long kGasBaselineDurationMs = 10000;
35 constexpr unsigned long kLedSelfTestIntervalMs = 500;
36 constexpr unsigned long kDangerAlarmDurationMs = 10000;
37 constexpr unsigned long kDetectionPauseDurationMs = 10000;
38 constexpr float kSen0565WarningVoltageThresholdV = 1.60f;
39 constexpr float kSen0565DangerVoltageThresholdV = 1.90f;
40 constexpr float kSen0565ExtremeVoltageThresholdV = 2.20f;
41 constexpr float kThermalRiskTempRiseC = 5.0f;
42 constexpr float kThermalRiskAbsoluteTempC = 55.0f;
43 constexpr uint8_t kGasLevelConfirmReads = 5;
44 constexpr bool kAlarmOutputActiveLow = true;
45 constexpr bool kFanRelayActiveLow = true;
46 constexpr float kGasBaselineFollowAlpha = 0.02f;
47 constexpr float kGasBaselineSettleDeltaV = 0.05f;
48 constexpr float kGasBaselineSettleRatio = 1.03f;
49
50 enum class GasLevel {

```

```

51     Safe,
52     Warning,
53     Danger,
54 };
55
56 struct GasSensorState {
57     const char* name;
58     int analogPin;
59     float warningVoltageThresholdV;
60     float dangerVoltageThresholdV;
61     float extremeVoltageThresholdV;
62     float baselineVoltage;
63     bool baselineReady;
64     int raw;
65     float voltage;
66     GasLevel level;
67     bool extreme;
68     bool detected;
69 };
70
71 TwoWire gOledWire = TwoWire(0);
72 TwoWire gBmeWire = TwoWire(1);
73 Adafruit_SSD1306 display(kScreenWidth, kScreenHeight, &gOledWire, -1);
74 Adafruit_BME280 gBme;
75
76 bool gDisplayReady = false;
77 bool gBmeReady = false;
78 uint8_t gDisplayAddress = 0;
79 uint8_t gBmeAddress = 0;
80
81 bool gWiFiConnected = false;
82 bool gWiFiConnecting = false;
83 unsigned long gWiFiConnectStartMs = 0;
84 unsigned long gLastWiFiAttemptMs = 0;
85
86 bool gLedOn = false;
87
88 bool gGasDetected = false;
89 bool gDangerAlarmActive = false;
90 bool gDetectionPaused = false;
91 bool gRedBlinkOn = false;
92 bool gAlarmCancelButtonLatched = false;
93 GasLevel gGasLevel = GasLevel::Safe;
94 GasLevel gLastGasLevel = GasLevel::Safe;
95 GasLevel gPendingGasLevel = GasLevel::Safe;
96 uint8_t gPendingGasLevelCount = 0;
97 unsigned long gStartMs = 0;
98 unsigned long gLastBlinkMs = 0;
99 unsigned long gLastRedBlinkMs = 0;
100 unsigned long gLastSensorReadMs = 0;
101 unsigned long gLastFirebaseHistoryMs = 0;
102 unsigned long gDangerAlarmStartMs = 0;
103 unsigned long gDetectionPauseStartMs = 0;
104 float gLastTemperatureC = NAN;
105 GasSensorState gSen0565Sensor = {"SEN0565",
106                                     kSen0565AnalogPin,
107                                     kSen0565WarningVoltageThresholdV,
108                                     kSen0565DangerVoltageThresholdV,
109                                     kSen0565ExtremeVoltageThresholdV,
110                                     0.0f,
111                                     false,
112                                     0,
113                                     0.0f,
114                                     GasLevel::Safe,
115                                     false,
116                                     false};
117
118 const char* gasLevelName(GasLevel level) {
119     switch (level) {
120         case GasLevel::Danger:
121             return "DANGER";
122         case GasLevel::Warning:
123             return "WARNING";
124         case GasLevel::Safe:
125             default:
126                 return "SAFE";
127     }
128 }
129
130 void setAlarmOutput(bool enabled) {
131     const uint8_t activeState = kAlarmOutputActiveLow ? LOW : HIGH;
132     const uint8_t inactiveState = kAlarmOutputActiveLow ? HIGH : LOW;
133     digitalWrite(kAlarmOutputPin, enabled ? activeState : inactiveState);

```

```

134 }
135
136 void setFanRelay(bool enabled) {
137     const uint8_t activeState = kFanRelayActiveLow ? LOW : HIGH;
138     const uint8_t inactiveState = kFanRelayActiveLow ? HIGH : LOW;
139     digitalWrite(kFanRelayPin, enabled ? activeState : inactiveState);
140 }
141
142 void setStatusLeds(bool greenOn, bool redOn, bool yellowOn);
143 void handleAlarmCancelButton(unsigned long now);
144 void updateDangerAlarm(unsigned long now);
145 void updateStatusLeds(unsigned long now);
146 void serviceAlarmControls(unsigned long now);
147 void startWiFiConnection(unsigned long now);
148 void serviceWiFi(unsigned long now);
149
150 void startDangerAlarm(unsigned long now) {
151     gDangerAlarmActive = true;
152     gDangerAlarmStartMs = now;
153     gRedBlinkOn = true;
154     gLastRedBlinkMs = now;
155     setAlarmOutput(true);
156     setFanRelay(true);
157 }
158
159 void stopDangerAlarm() {
160     gDangerAlarmActive = false;
161     gGasDetected = false;
162     gGasLevel = GasLevel::Safe;
163     gPendingGasLevel = GasLevel::Safe;
164     gPendingGasLevelCount = 0;
165     gRedBlinkOn = false;
166     setAlarmOutput(false);
167     setFanRelay(false);
168     setStatusLeds(true, false, false);
169 }
170
171 void startDetectionPause(unsigned long now) {
172     stopDangerAlarm();
173     gDetectionPaused = true;
174     gDetectionPauseStartMs = now;
175 }
176
177 void updateDetectionPause(unsigned long now) {
178     if (!gDetectionPaused) {
179         return;
180     }
181
182     if (now - gDetectionPauseStartMs >= kDetectionPauseDurationMs) {
183         gDetectionPaused = false;
184         gLastSensorReadMs = 0;
185         Serial.println("Detection pause ended; monitoring resumed");
186         return;
187     }
188 }
189
190 void updateDangerAlarm(unsigned long now) {
191     if (!gDangerAlarmActive) {
192         setAlarmOutput(false);
193         setFanRelay(false);
194         return;
195     }
196
197     if (now - gDangerAlarmStartMs >= kDangerAlarmDurationMs) {
198         stopDangerAlarm();
199         return;
200     }
201
202     setAlarmOutput(true);
203     setFanRelay(true);
204 }
205
206 void setStatusLeds(bool greenOn, bool redOn, bool yellowOn) {
207     digitalWrite(kGreenLedPin, greenOn ? HIGH : LOW);
208     digitalWrite(kRedLedPin, redOn ? HIGH : LOW);
209     digitalWrite(kYellowLedPin, yellowOn ? HIGH : LOW);
210 }
211
212 void runLedSelfTest() {
213     setStatusLeds(false, false, false);
214     delay(100);
215     Serial.println("LED self-test: green -> yellow -> red");
216 }

```

```

217   setStatusLeds(true, false, false);
218   delay(kLedSelfTestIntervalMs);
219   setStatusLeds(false, false, true);
220   delay(kLedSelfTestIntervalMs);
221   setStatusLeds(false, true, false);
222   delay(kLedSelfTestIntervalMs);
223   setStatusLeds(false, false, false);
224 }
225
226 void startWiFiConnection(unsigned long now) {
227     if (WiFi.status() == WL_CONNECTED || gWiFiConnecting) {
228         return;
229     }
230
231     Serial.printf("Starting WiFi connection to SSID='%s'...\r\n", WIFI_SSID);
232     WiFi.mode(WIFI_STA);
233     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
234     gWiFiConnecting = true;
235     gWiFiConnectStartMs = now;
236     gLastWiFiAttemptMs = now;
237 }
238
239 void serviceWiFi(unsigned long now) {
240     const wl_status_t status = WiFi.status();
241
242     if (status == WL_CONNECTED) {
243         if (!gWiFiConnected) {
244             Serial.printf("WiFi connected, IP=%s\r\n", WiFi.localIP().toString().c_str());
245         }
246         gWiFiConnected = true;
247         gWiFiConnecting = false;
248         return;
249     }
250
251     if (gWiFiConnected) {
252         Serial.println("WiFi disconnected. Local monitoring continues; Firebase upload paused.");
253     }
254     gWiFiConnected = false;
255
256     if (gWiFiConnecting) {
257         if (now - gWiFiConnectStartMs >= kWifiConnectTimeoutMs) {
258             Serial.println("WiFi connection timed out. Local monitoring continues; will retry later.");
259             WiFi.disconnect(false);
260             gWiFiConnecting = false;
261         }
262         return;
263     }
264
265     if (gLastWiFiAttemptMs == 0 || now - gLastWiFiAttemptMs >= kWifiRetryIntervalMs) {
266         startWiFiConnection(now);
267     }
268 }
269
270 float gasRatio(const GasSensorState& sensor);
271 float gasNormalizedRatio(const GasSensorState& sensor);
272 float gasDeltaMagnitude(const GasSensorState& sensor);
273
274 float estimatePpm(const GasSensorState& sensor) {
275     if (!sensor.baselineReady || sensor.baselineVoltage <= 0.0f) {
276         return 0.0f;
277     }
278
279     const float ratio = gasNormalizedRatio(sensor);
280     if (ratio <= 1.0f) {
281         return 0.0f;
282     }
283     if (ratio <= 2.2f) {
284         return (ratio - 1.0f) * 250.0f;
285     }
286     return 300.0f + (ratio - 2.2f) * 500.0f;
287 }
288
289 String jsonFloat(float value, unsigned int decimalPlaces) {
290     if (isnan(value) || isinf(value)) {
291         return "null";
292     }
293     return String(value, decimalPlaces);
294 }
295
296 String firebasePayload(float ppm,
297                       float rawPpm,
298                       float gasVoltage,
299                       float temperatureC,

```

```

300         float humidityPct,
301         const char* state,
302         bool fan,
303         bool buzzer,
304         bool thermalRisk) {
305     String payload = "{";
306     payload += "\"ppm_compensated\": " + jsonFloat(ppm, 1) + ",";
307     payload += "\"ppm_raw\": " + jsonFloat(rawPpm, 1) + ",";
308     payload += "\"voltage\": " + jsonFloat(gasVoltage, 3) + ",";
309     payload += "\"temperature\": " + jsonFloat(temperatureC, 1) + ",";
310     payload += "\"humidity\": " + jsonFloat(humidityPct, 1) + ",";
311     payload += "\"state\": \"" + String(state) + "\"";
312     payload += "\"fan\": " + String(fan ? "true" : "false") + ",";
313     payload += "\"buzzer\": " + String(buzzer ? "true" : "false") + ",";
314     payload += "\"thermal_risk\": " + String(thermalRisk ? "true" : "false") + ",";
315     payload += "\"timestamp\": {\".sv\": \"timestamp\"}";
316     payload += "}";
317     return payload;
318 }
319
320 bool uploadReadingToFirebase(const char* node,
321                             bool append,
322                             float ppmCompensated,
323                             float ppmRaw,
324                             float gasVoltage,
325                             float temperatureC,
326                             float humidityPct,
327                             const char* state,
328                             bool fan,
329                             bool buzzer,
330                             bool thermalRisk) {
331     if (WiFi.status() != WL_CONNECTED) {
332         return false;
333     }
334
335     WiFiClientSecure client;
336     client.setInsecure();
337     HTTPClient http;
338     const String url = String("https://") + FIREBASE_HOST + "/leaksense/" + node + ".json?auth=" + FIREBASE_AUTH;
339
340     if (!http.begin(client, url)) {
341         Serial.println("Firebase: begin failed");
342         return false;
343     }
344
345     http.addHeader("Content-Type", "application/json");
346     const String payload = firebasePayload(ppmCompensated,
347                                           ppmRaw,
348                                           gasVoltage,
349                                           temperatureC,
350                                           humidityPct,
351                                           state,
352                                           fan,
353                                           buzzer,
354                                           thermalRisk);
355     const int httpCode = append ? http.POST(payload) : http.PUT(payload);
356
357     if (httpCode <= 0) {
358         Serial.printf("Firebase: request failed (%d)\r\n", httpCode);
359         http.end();
360         return false;
361     }
362
363     if (httpCode >= 200 && httpCode < 300) {
364         Serial.printf("Firebase: uploaded %s reading, code=%d\r\n", node, httpCode);
365         http.end();
366         return true;
367     }
368
369     Serial.printf("Firebase: upload error %d -> %s\r\n", httpCode, http.errorToString(httpCode).c_str());
370     http.end();
371     return false;
372 }
373
374 bool uploadLatestToFirebase(float ppmCompensated,
375                             float ppmRaw,
376                             float gasVoltage,
377                             float temperatureC,
378                             float humidityPct,
379                             const char* state,
380                             bool fan,
381                             bool buzzer,
382                             bool thermalRisk) {

```

```

383     return uploadReadingToFirebase("latest",
384                                   false,
385                                   ppmCompensated,
386                                   ppmRaw,
387                                   gasVoltage,
388                                   temperatureC,
389                                   humidityPct,
390                                   state,
391                                   fan,
392                                   buzzer,
393                                   thermalRisk);
394 }
395
396 bool uploadHistoryToFirebase(float ppmCompensated,
397                             float ppmRaw,
398                             float gasVoltage,
399                             float temperatureC,
400                             float humidityPct,
401                             const char* state,
402                             bool fan,
403                             bool buzzer,
404                             bool thermalRisk) {
405     return uploadReadingToFirebase("history",
406                                   true,
407                                   ppmCompensated,
408                                   ppmRaw,
409                                   gasVoltage,
410                                   temperatureC,
411                                   humidityPct,
412                                   state,
413                                   fan,
414                                   buzzer,
415                                   thermalRisk);
416 }
417
418 void updateStatusLeds(unsigned long now) {
419     if (gDetectionPaused) {
420         updateDetectionPause(now);
421         return;
422     }
423
424     if (gDangerAlarmActive) {
425         if (now - gLastRedBlinkMs >= kRedBlinkIntervalMs) {
426             gLastRedBlinkMs = now;
427             gRedBlinkOn = !gRedBlinkOn;
428         }
429         setStatusLeds(false, gRedBlinkOn, false);
430         return;
431     }
432
433     switch (gGasLevel) {
434         case GasLevel::Danger:
435             setStatusLeds(false, true, false);
436             break;
437         case GasLevel::Warning:
438             setStatusLeds(false, false, true);
439             break;
440         case GasLevel::Safe:
441             default:
442                 setStatusLeds(true, false, false);
443                 break;
444     }
445 }
446
447 void serviceAlarmControls(unsigned long now) {
448     handleAlarmCancelButton(now);
449     updateDangerAlarm(now);
450     updateStatusLeds(now);
451 }
452
453 void handleAlarmCancelButton(unsigned long now) {
454     const bool pressed = digitalRead(kAlarmCancelButtonPin) == LOW;
455     if (!pressed) {
456         gAlarmCancelButtonLatched = false;
457         return;
458     }
459
460     if (gAlarmCancelButtonLatched) {
461         return;
462     }
463
464     gAlarmCancelButtonLatched = true;
465     Serial.println("GPIO14 button pressed: alarm stopped; gas detection paused for 10 seconds");

```

```

466     startDetectionPause(now);
467 }
468
469 GasLevel confirmGasLevel(GasLevel newLevel, bool extremeDanger) {
470     if (newLevel == GasLevel::Safe) {
471         gPendingGasLevel = GasLevel::Safe;
472         gPendingGasLevelCount = 0;
473         return GasLevel::Safe;
474     }
475
476     if (newLevel == GasLevel::Warning) {
477         gPendingGasLevel = GasLevel::Warning;
478         gPendingGasLevelCount = 0;
479         return GasLevel::Warning;
480     }
481
482     if (extremeDanger) {
483         gPendingGasLevel = GasLevel::Danger;
484         gPendingGasLevelCount = 0;
485         return GasLevel::Danger;
486     }
487
488     if (gPendingGasLevel != GasLevel::Danger) {
489         gPendingGasLevel = GasLevel::Danger;
490         gPendingGasLevelCount = 1;
491     } else if (gPendingGasLevelCount < kGasLevelConfirmReads) {
492         ++gPendingGasLevelCount;
493     }
494     if (gPendingGasLevelCount >= kGasLevelConfirmReads) {
495         gPendingGasLevelCount = 0;
496         return GasLevel::Danger;
497     }
498
499     return GasLevel::Warning;
500 }
501
502 void scanI2CBus(TwoWire& bus, const char* label) {
503     Serial.printf("Scanning %s I2C bus...\r\n", label);
504     for (uint8_t address = 1; address < 127; ++address) {
505         bus.beginTransmission(address);
506         if (bus.endTransmission() == 0) {
507             Serial.printf("%s bus device found at 0x%02X\r\n", label, address);
508         }
509     }
510 }
511
512 bool initDisplay() {
513     for (uint8_t address : kOledAddresses) {
514         if (display.begin(SSD1306_SWITCHCAPVCC, address)) {
515             gDisplayAddress = address;
516             return true;
517         }
518     }
519     return false;
520 }
521
522 bool initBme() {
523     for (uint8_t address : kBmeAddresses) {
524         if (gBme.begin(address, &gBmeWire)) {
525             gBmeAddress = address;
526             return true;
527         }
528     }
529     return false;
530 }
531
532 float readAnalogVoltage(int analogPin) {
533     uint32_t total = 0;
534     for (uint8_t i = 0; i < kGasSamplesPerRead; ++i) {
535         total += analogRead(analogPin);
536         serviceAlarmControls(millis());
537         delay(2);
538     }
539
540     const float raw = total / static_cast<float>(kGasSamplesPerRead);
541     return raw * 3.3f / 4095.0f;
542 }
543
544 void updateGasBaseline(GasSensorState& sensor) {
545     if (sensor.baselineVoltage <= 0.0f) {
546         sensor.baselineVoltage = sensor.voltage;
547         return;
548     }

```

```

549
550 if (!sensor.baselineReady) {
551     sensor.baselineVoltage = (sensor.baselineVoltage * 0.85f) + (sensor.voltage * 0.15f);
552     if (millis() - gStartMs >= kGasBaselineDurationMs) {
553         sensor.baselineReady = true;
554         Serial.printf("%s baseline ready: %.3f V\r\n", sensor.name, sensor.baselineVoltage);
555     }
556     return;
557 }
558
559 if (gasDeltaMagnitude(sensor) < kGasBaselineSettleDeltaV &&
560     gasNormalizedRatio(sensor) < kGasBaselineSettleRatio) {
561     sensor.baselineVoltage =
562         (sensor.baselineVoltage * (1.0f - kGasBaselineFollowAlpha)) +
563         (sensor.voltage * kGasBaselineFollowAlpha);
564 }
565 }
566
567 float gasRatio(const GasSensorState& sensor) {
568     return sensor.baselineVoltage > 0.01f ? sensor.voltage / sensor.baselineVoltage : 0.0f;
569 }
570
571 float gasNormalizedRatio(const GasSensorState& sensor) {
572     const float ratio = gasRatio(sensor);
573     if (ratio <= 0.0f) {
574         return 1.0f;
575     }
576     return ratio >= 1.0f ? ratio : (1.0f / ratio);
577 }
578
579 float gasDeltaMagnitude(const GasSensorState& sensor) {
580     return fabsf(sensor.voltage - sensor.baselineVoltage);
581 }
582
583 GasLevel evaluateGasLevel(const GasSensorState& sensor) {
584     if (sensor.voltage >= sensor.extremeVoltageThresholdV) {
585         return GasLevel::Danger;
586     }
587     if (sensor.voltage >= sensor.dangerVoltageThresholdV) {
588         return GasLevel::Danger;
589     }
590     if (sensor.voltage >= sensor.warningVoltageThresholdV) {
591         return GasLevel::Warning;
592     }
593     return GasLevel::Safe;
594 }
595
596 bool hasThermalRisk(float temperatureC) {
597     if (!isfinite(temperatureC)) {
598         return false;
599     }
600
601     const bool highTemperature = temperatureC >= kThermalRiskAbsoluteTempC;
602     const bool rapidRise = isfinite(gLastTemperatureC) &&
603         (temperatureC - gLastTemperatureC) >= kThermalRiskTempRiseC;
604
605     return highTemperature || rapidRise;
606 }
607
608 void sampleGasSensor(GasSensorState& sensor) {
609     sensor.raw = analogRead(sensor.analogPin);
610     sensor.voltage = readAnalogVoltage(sensor.analogPin);
611     updateGasBaseline(sensor);
612     sensor.extreme = sensor.baselineReady &&
613         sensor.voltage >= sensor.extremeVoltageThresholdV;
614     sensor.level = evaluateGasLevel(sensor);
615     sensor.detected = sensor.level == GasLevel::Danger;
616 }
617
618 void printGasSensor(const GasSensorState& sensor) {
619     if (sensor.baselineReady) {
620         const float signedDelta = sensor.voltage - sensor.baselineVoltage;
621         Serial.printf("%s -> raw=%d, V=%.3f, baseline=%.3f, delta=%.3f, absDelta=%.3f, ratio=%.2f, state=%s, extreme=%s\r\n",
622             sensor.name,
623             sensor.raw,
624             sensor.voltage,
625             sensor.baselineVoltage,
626             signedDelta,
627             gasDeltaMagnitude(sensor),
628             gasNormalizedRatio(sensor),
629             gasLevelName(sensor.level),
630             sensor.extreme ? "YES" : "no");
631     } else {

```

```

632     Serial.printf("%s baselining -> raw=%d, V=%.3f\r\n",
633                   sensor.name,
634                   sensor.raw,
635                   sensor.voltage);
636 }
637 }
638
639 void drawStatusScreen(float temperatureC,
640                      float humidityPct,
641                      float pressureHpa,
642                      const GasSensorState& sensor,
643                      GasLevel confirmedLevel,
644                      bool gasDetected,
645                      bool fanOn) {
646     display.clearDisplay();
647     display.setTextSize(1);
648     display.setTextColor(SSD1306_WHITE);
649     display.setCursor(0, 0);
650     display.println("LeakSense SEN0565");
651
652     if (gBmeReady) {
653         display.printf("T:%.1fC H:%.0f%% P:%.0f\r\n", temperatureC, humidityPct, pressureHpa);
654     } else {
655         display.println("BME280 not found");
656     }
657
658 }
659
660 display.printf("Raw:%4d V:%.2f\r\n", sensor.raw, sensor.voltage);
661 if (sensor.baselineReady) {
662     display.printf("Base:%.2f R:%.2f\r\n", sensor.baselineVoltage, gasNormalizedRatio(sensor));
663     display.printf("State:%s\r\n", gasLevelName(confirmedLevel));
664     display.printf("Alarm:%s Fan:%s\r\n", gasDetected ? "ON" : "off", fanOn ? "ON" : "off");
665 }
666
667 } else {
668     const unsigned long elapsedMs = millis() - gStartMs;
669     const unsigned long cappedElapsedMs =
670         elapsedMs > kGasBaselineDurationMs ? kGasBaselineDurationMs : elapsedMs;
671     const unsigned long remainingSec = (kGasBaselineDurationMs - cappedElapsedMs) / 1000;
672     display.println("Baselining SEN0565");
673
674     display.printf("Time left: %lus\r\n", remainingSec);
675 }
676
677 display.display();
678 }
679
680 void setup() {
681     Serial.begin(115200);
682     pinMode(kLedPin, OUTPUT);
683     pinMode(kGreenLedPin, OUTPUT);
684     pinMode(kRedLedPin, OUTPUT);
685     pinMode(kYellowLedPin, OUTPUT);
686     pinMode(kAlarmOutputPin, OUTPUT);
687     pinMode(kFanRelayPin, OUTPUT);
688     pinMode(kAlarmCancelButtonPin, INPUT_PULLUP);
689     gAlarmCancelButtonLatched = false;
690     setAlarmOutput(false);
691     setFanRelay(false);
692     runLedSelfTest();
693     updateStatusLeds(millis());
694
695     analogReadResolution(12);
696     analogSetPinAttenuation(kSen0565AnalogPin, ADC_11db);
697     gStartMs = millis();
698     delay(200);
699
700
701
702
703
704
705     Serial.printf("OLED test starting (SDA=%u, SCL=%u)\r\n", kOledSdaPin, kOledSclPin);
706     gOledWire.begin(kOledSdaPin, kOledSclPin);
707     scanI2CBus(gOledWire, "OLED");
708
709     gDisplayReady = initDisplay();
710     if (gDisplayReady) {
711         Serial.printf("OLED initialized at 0x%02X\r\n", gDisplayAddress);
712     } else {
713         Serial.println("OLED init failed. Check VCC/GND/SDA/SCL and address.");
714     }

```

```

715 Serial.printf("BME280 test starting (SDA=%u, SCL=%u)\r\n", kBmeSdaPin, kBmeSclPin);
716 gBmeWire.begin(kBmeSdaPin, kBmeSclPin);
717 scanI2CBus(gBmeWire, "BME280");
718
719 gBmeReady = initBme();
720 if (gBmeReady) {
721     Serial.printf("BME280 initialized at 0x%02X\r\n", gBmeAddress);
722 } else {
723     Serial.println("BME280 init failed. Check VCC/GND/SDA/SCL and ADR/CS pins.");
724 }
725
726 Serial.printf("%s test starting (A=GPIO%d)\r\n", gSen0565Sensor.name, gSen0565Sensor.analogPin);
727 Serial.println("SEN0565 alarm contribution: enabled");
728 Serial.println("Keep SEN0565 in clean air for the first 10 seconds.");
729 Serial.println("Warm SEN0565 for at least 5 minutes before trusting its thresholds.");
730
731 startWiFiConnection(millis());
732 Serial.println("WiFi will connect in background. Local monitoring is available offline.");
733 }
734
735 void loop() {
736     const unsigned long now = millis();
737     serviceAlarmControls(now);
738     serviceWiFi(now);
739
740     if (now - gLastBlinkMs >= kBlinkIntervalMs) {
741         gLastBlinkMs = now;
742         gLedOn = !gLedOn;
743         digitalWrite(kLedPin, gLedOn ? HIGH : LOW);
744     }
745
746     if (!gDetectionPaused && now - gLastSensorReadMs >= kSensorReadIntervalMs) {
747         gLastSensorReadMs = now;
748
749         float temperatureC = NAN;
750         float humidityPct = NAN;
751         float pressureHpa = NAN;
752         sampleGasSensor(gSen0565Sensor);
753         if (gDetectionPaused) {
754             return;
755         }
756         if (gBmeReady) {
757             temperatureC = gBme.readTemperature();
758             humidityPct = gBme.readHumidity();
759             pressureHpa = gBme.readPressure() / 100.0f;
760
761             Serial.printf("BME280 0x%02X -> T=%.2f C, H=%.2f %, P=%.2f hPa\r\n",
762                 gBmeAddress, temperatureC, humidityPct, pressureHpa);
763         } else {
764             Serial.println("BME280 not ready");
765         }
766     }
767
768     gLastGasLevel = gGasLevel;
769     gGasLevel = confirmGasLevel(gSen0565Sensor.level, gSen0565Sensor.extreme);
770     const bool thermalRisk = hasThermalRisk(temperatureC);
771     if (thermalRisk) {
772         gGasLevel = GasLevel::Danger;
773         Serial.println("Thermal risk escalation: abnormal temperature rise/high temperature");
774     }
775     gGasDetected = gGasLevel == GasLevel::Danger;
776     if (gGasDetected && gLastGasLevel != GasLevel::Danger) {
777         startDangerAlarm(now);
778     }
779     updateDangerAlarm(now);
780     updateStatusLeds(now);
781     if (isfinite(temperatureC)) {
782         gLastTemperatureC = temperatureC;
783     }
784
785     printGasSensor(gSen0565Sensor);
786     Serial.printf("Alarm source -> SEN0565 raw=%s, confirmed=%s, alarm=%s, fan=%s\r\n",
787         gasLevelName(gSen0565Sensor.level),
788         gasLevelName(gGasLevel),
789         gDangerAlarmActive ? "ON" : "off",
790         gDangerAlarmActive ? "ON" : "off");
791
792
793
794
795
796
797

```

```

798
799     const float ppmCompensated = estimatePpm(gSen0565Sensor);
800     const float ppmRaw = gSen0565Sensor.raw * 0.25f;
801     const char* stateName = gasLevelName(gGasLevel);
802     const bool fanOn = gDangerAlarmActive;
803     const bool buzzerOn = gDangerAlarmActive;
804     const bool shouldUploadAlertHistory =
805         gGasLevel == GasLevel::Warning || gGasLevel == GasLevel::Danger;
806
807     if (WiFi.status() == WL_CONNECTED) {
808         uploadLatestToFirebase(ppmCompensated,
809                                ppmRaw,
810                                gSen0565Sensor.voltage,
811                                temperatureC,
812                                humidityPct,
813                                stateName,
814                                fanOn,
815                                buzzerOn,
816                                thermalRisk);
817
818         if (shouldUploadAlertHistory ||
819             gLastFirebaseHistoryMs == 0 ||
820             now - gLastFirebaseHistoryMs >= kFirebaseHistoryIntervalMs) {
821             if (uploadHistoryToFirebase(ppmCompensated,
822                                         ppmRaw,
823                                         gSen0565Sensor.voltage,
824                                         temperatureC,
825                                         humidityPct,
826                                         stateName,
827                                         fanOn,
828                                         buzzerOn,
829                                         thermalRisk)) {
830                 gLastFirebaseHistoryMs = now;
831             }
832         }
833     }
834
835     if (gDisplayReady) {
836         drawStatusScreen(temperatureC,
837                          humidityPct,
838                          pressureHpa,
839                          gSen0565Sensor,
840                          gGasLevel,
841                          gDangerAlarmActive,
842                          gDangerAlarmActive);
843     }
844 }
845

```

B. PlatformIO Configuration

Listing 2. PlatformIO configuration: firmware/blinkng/platformio.ini.

```

1 [env:firebeetle2_esp32e]
2 platform = espressif32
3 board = dfrobot_firebeetle2_esp32e
4 framework = arduino
5 monitor_speed = 115200
6 lib_deps =
7     adafruit/Adafruit GFX Library @ ^1.12.1
8     adafruit/Adafruit SSD1306 @ ^2.5.15
9     adafruit/Adafruit BME280 Library @ ^2.3.0
10
11 LeakSense - Smart LPG Gas Leakage Detection and Automatic Safety System

```

C. Credential Template

Listing 3. Credential template: firmware/blinkng/src/secrets.h.example.

```

1 #pragma once
2
3 #define WIFI_SSID "YOUR_WIFI_NAME"
4 #define WIFI_PASSWORD "YOUR_WIFI_PASSWORD"
5
6 #define FIREBASE_HOST "YOUR_PROJECT-default-rtdb.firebaseio.com"
7 #define FIREBASE_AUTH "YOUR_REALTIME_DATABASE_SECRET"

```

D. Dashboard HTML

Listing 4. Dashboard HTML: dashboard/index.html.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>LeakSense -- LPG Gas Detection Dashboard</title>
7
8   <!-- Chart.js -->
9   <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
10
11   <!-- Firebase -->
12   <script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-app-compat.js"></script>
13   <script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-database-compat.js"></script>
14
15   <link rel="stylesheet" href="css/style.css" />
16 </head>
17 <body>
18
19 <div class="app">
20
21   <!-- -- Top bar ----- -->
22   <header class="topbar">
23     <div class="topbar__left">
24       <span class="topbar__title">LeakSense</span>
25       <span class="topbar__sub">Group 27 · Smart LPG Gas Detection System</span>
26     </div>
27     <div class="topbar__right">
28       <span class="topbar__sync" id="lastSync">Last sync: --</span>
29       <button class="notification-btn" type="button" id="enableNotificationsBtn" hidden>
30         Enable notifications
31       </button>
32       <span class="status-badge safe" id="statusBadge">Safe</span>
33     </div>
34   </header>
35
36   <!-- -- State notification bar ----- -->
37   <div class="state-bar safe" id="stateBar">
38     System operating normally. Gas levels within safe range.
39   </div>
40
41   <!-- -- Metric cards ----- -->
42   <section class="metrics">
43     <div class="card metric-card">
44       <p class="metric-card__label">Gas voltage</p>
45       <p class="metric-card__value" id="ppmVal">-- V</p>
46       <p class="metric-card__sub" id="ppmSub">Sensor output</p>
47     </div>
48     <div class="card metric-card">
49       <p class="metric-card__label">Temperature</p>
50       <p class="metric-card__value" id="tempVal">--°C</p>
51       <p class="metric-card__sub">BME280 reading</p>
52     </div>
53     <div class="card metric-card">
54       <p class="metric-card__label">Humidity</p>
55       <p class="metric-card__value" id="humVal">--%</p>
56       <p class="metric-card__sub">BME280 reading</p>
57     </div>
58     <div class="card metric-card">
59       <p class="metric-card__label">Alerts today</p>
60       <p class="metric-card__value" id="alertCount">0</p>
61       <p class="metric-card__sub">Warning + Danger + Extreme events</p>
62     </div>
63   </section>
64
65   <!-- -- Live main grid ----- -->
66   <div class="main-grid view-panel" id="liveView">
67
68     <!-- Live trend chart -->
69     <div class="card chart-card">
70       <div class="card__header">
71         <p class="card__title">Gas voltage -- last 60 readings (live)</p>
72         <button class="view-toggle-btn" type="button" id="showHistoryBtn">History</button>
73       </div>
74       <canvas id="trendChart"></canvas>
75     </div>
76
77     <!-- Device states + alert log -->
78     <div class="right-col">
```

```

79
80 <!-- Device states -->
81 <div class="card">
82   <p class="card__title">Device states</p>
83   <div class="device-list">
84
85     <div class="device-row">
86       <div>
87         <p class="device-row__name">Exhaust fan</p>
88         <p class="device-row__sub">Relay -- Danger / Extreme</p>
89       </div>
90       <span class="pill" id="fanState">off</span>
91     </div>
92
93     <div class="device-row">
94       <div>
95         <p class="device-row__name">Buzzer</p>
96         <p class="device-row__sub">Danger / Extreme -- 10s cycle / button</p>
97       </div>
98       <span class="pill" id="buzzerState">off</span>
99     </div>
100
101     <div class="device-row">
102       <div>
103         <p class="device-row__name">Green LED</p>
104         <p class="device-row__sub">Safe state indicator</p>
105       </div>
106       <span class="pill pill--on" id="greenLed">on</span>
107     </div>
108
109     <div class="device-row">
110       <div>
111         <p class="device-row__name">Yellow LED</p>
112         <p class="device-row__sub">Warning state indicator</p>
113       </div>
114       <span class="pill" id="yellowLed">off</span>
115     </div>
116
117     <div class="device-row">
118       <div>
119         <p class="device-row__name">Red LED</p>
120         <p class="device-row__sub">Danger / Extreme indicator</p>
121       </div>
122       <span class="pill" id="redLed">off</span>
123     </div>
124
125     <div class="device-row device-row--last">
126       <div>
127         <p class="device-row__name">Cloud sync</p>
128         <p class="device-row__sub">Firebase · every 2s</p>
129       </div>
130       <span class="pill pill--on" id="cloudSync">live</span>
131     </div>
132
133   </div>
134 </div>
135
136 <!-- Alert log -->
137 <div class="card alert-card">
138   <p class="card__title">Alert log</p>
139   <div class="alert-log" id="alertLog">
140     <p class="alert-log__empty">No alerts yet -- system safe.</p>
141   </div>
142 </div>
143
144 </div>
145 </div><!-- /.main-grid -->
146
147 <!-- -- 24-hour history ----- -->
148 <div class="history-section view-panel" id="historyView">
149
150   <!-- 24hr chart -->
151   <div class="card history-chart-card">
152     <div class="card__header">
153       <p class="card__title">24-hour history -- Gas · Temperature · Humidity</p>
154       <button class="view-toggle-btn" type="button" id="showLiveBtn">Live</button>
155     </div>
156     <canvas id="historyChart"></canvas>
157   </div>
158
159   <!-- 24hr data table -->
160   <div class="card">
161     <p class="card__title">24-hour reading log (every 5 minutes)</p>

```

```

162     <div class="history-table-wrap">
163       <p class="history-empty" id="historyEmpty">Loading history...</p>
164       <table class="history-table">
165         <thead>
166           <tr>
167             <th>Time</th>
168             <th>Gas (V)</th>
169             <th>Temp</th>
170             <th>Humidity</th>
171             <th>State</th>
172           </tr>
173         </thead>
174         <tbody id="historyTableBody"></tbody>
175       </table>
176     </div>
177   </div>
178
179   </div><!-- /.history-section -->
180
181 </div><!-- /.app -->
182
183 <!-- JS -- order matters -->
184 <script src="js/mock.js"></script>
185 <script src="js/charts.js"></script>
186 <script src="js/secrets.js"></script>
187 <script src="js/firebase.js"></script>
188 <script src="js/dashboard.js"></script>
189
190 </body>
191 </html>

```

E. Dashboard CSS

Listing 5. Dashboard CSS: dashboard/css/style.css.

```

1  /* -- Reset & base ----- */
2  *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }
3
4  body {
5    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
6    background: #f3f4f6;
7    color: #111827;
8    font-size: 14px;
9    line-height: 1.5;
10 }
11
12 /* -- Layout ----- */
13 .app {
14   max-width: 1100px;
15   margin: 0 auto;
16   padding: 1.25rem;
17 }
18
19 /* -- Top bar ----- */
20 .topbar {
21   display: flex;
22   align-items: center;
23   justify-content: space-between;
24   margin-bottom: 1rem;
25   padding-bottom: 1rem;
26   border-bottom: 1px solid #e5e7eb;
27 }
28 .topbar__left { display: flex; flex-direction: column; gap: 2px; }
29 .topbar__right { display: flex; align-items: center; gap: 12px; }
30 .topbar__title { font-size: 16px; font-weight: 600; color: #111827; }
31 .topbar__sub { font-size: 12px; color: #6b7280; }
32 .topbar__sync { font-size: 12px; color: #9ca3af; }
33
34 .notification-btn {
35   border: 1px solid #c7d2fe;
36   border-radius: 8px;
37   background: #eef2ff;
38   color: #3730a3;
39   cursor: pointer;
40   font: inherit;
41   font-size: 12px;
42   font-weight: 600;
43   line-height: 1;
44   padding: 7px 10px;
45   transition: background 0.15s, border-color 0.15s, color 0.15s;
46   white-space: nowrap;

```

```

47 }
48 .notification-btn:hover {
49     background: #e0e7ff;
50     border-color: #a5b4fc;
51 }
52 .notification-btn:focus-visible {
53     outline: 3px solid rgba(99, 102, 241, 0.25);
54     outline-offset: 2px;
55 }
56 .notification-btn:disabled {
57     background: #f3f4f6;
58     border-color: #e5e7eb;
59     color: #6b7280;
60     cursor: default;
61 }
62
63 /* -- Status badge ----- */
64 .status-badge {
65     display: inline-flex;
66     align-items: center;
67     gap: 6px;
68     padding: 4px 14px;
69     border-radius: 20px;
70     font-size: 13px;
71     font-weight: 500;
72     transition: background 0.3s, color 0.3s;
73 }
74 .status-badge::before {
75     content: '';
76     display: inline-block;
77     width: 7px;
78     height: 7px;
79     border-radius: 50%;
80     background: currentColor;
81 }
82 .status-badge.safe { background: #d1fae5; color: #065f46; }
83 .status-badge.warning { background: #fef3c7; color: #92400e; }
84 .status-badge.danger { background: #fee2e2; color: #991b1b; }
85 .status-badge.extreme { background: #7f1d1d; color: #fff7ed; }
86
87 /* -- State notification bar ----- */
88 .state-bar {
89     padding: 10px 16px;
90     border-radius: 10px;
91     font-size: 13px;
92     font-weight: 500;
93     margin-bottom: 1rem;
94     border: 1px solid transparent;
95     transition: background 0.3s, color 0.3s;
96 }
97 .state-bar.safe { background: #d1fae5; color: #065f46; border-color: #6ee7b7; }
98 .state-bar.warning { background: #fef3c7; color: #92400e; border-color: #fcd34d; }
99 .state-bar.danger { background: #fee2e2; color: #991b1b; border-color: #fca5a5; }
100 .state-bar.extreme { background: #7f1d1d; color: #fff7ed; border-color: #ef4444; }
101
102 /* -- Card ----- */
103 .card {
104     background: #ffffff;
105     border: 1px solid #e5e7eb;
106     border-radius: 12px;
107     padding: 1.1rem 1.25rem;
108 }
109 .card__title {
110     font-size: 12px;
111     font-weight: 500;
112     color: #6b7280;
113     margin-bottom: 0.75rem;
114     text-transform: uppercase;
115     letter-spacing: 0.04em;
116 }
117 .card__header {
118     display: flex;
119     align-items: center;
120     justify-content: space-between;
121     gap: 12px;
122     margin-bottom: 0.75rem;
123 }
124 .card__header .card__title { margin-bottom: 0; }
125
126 .view-panel[hidden] { display: none; }
127
128 .view-toggle-btn {
129     border: 1px solid #bfdbfe;

```

```

130 border-radius: 8px;
131 background: #eff6ff;
132 color: #1d4ed8;
133 cursor: pointer;
134 font: inherit;
135 font-size: 12px;
136 font-weight: 600;
137 line-height: 1;
138 padding: 7px 12px;
139 transition: background 0.15s, border-color 0.15s, color 0.15s;
140 white-space: nowrap;
141 }
142 .view-toggle-btn:hover {
143   background: #d8eaf8;
144   border-color: #93c5fd;
145   color: #1e40af;
146 }
147 .view-toggle-btn:focus-visible {
148   outline: 3px solid rgba(59, 130, 246, 0.25);
149   outline-offset: 2px;
150 }
151
152 /* -- Metric cards ----- */
153 .metrics {
154   display: grid;
155   grid-template-columns: repeat(4, 1fr);
156   gap: 12px;
157   margin-bottom: 1rem;
158 }
159 .metric-card__label { font-size: 12px; color: #6b7280; margin-bottom: 4px; }
160 .metric-card__value { font-size: 24px; font-weight: 500; color: #111827; }
161 .metric-card__sub { font-size: 11px; color: #9ca3af; margin-top: 3px; }
162
163 /* -- Main grid ----- */
164 .main-grid {
165   display: grid;
166   grid-template-columns: 1fr 340px;
167   gap: 12px;
168   align-items: start;
169   margin-bottom: 1rem;
170 }
171
172 /* -- Chart card ----- */
173 .chart-card canvas {
174   width: 100% !important;
175   height: 200px !important;
176 }
177
178 /* -- Right column ----- */
179 .right-col {
180   display: flex;
181   flex-direction: column;
182   gap: 12px;
183 }
184
185 /* -- Device list ----- */
186 .device-list { display: flex; flex-direction: column; }
187
188 .device-row {
189   display: flex;
190   align-items: center;
191   justify-content: space-between;
192   padding: 9px 0;
193   border-bottom: 1px solid #f3f4f6;
194 }
195 .device-row--last { border-bottom: none; }
196 .device-row__name { font-size: 13px; color: #111827; }
197 .device-row__sub { font-size: 11px; color: #9ca3af; margin-top: 1px; }
198
199 /* -- Pills ----- */
200 .pill {
201   display: inline-block;
202   padding: 3px 11px;
203   border-radius: 20px;
204   font-size: 11px;
205   font-weight: 500;
206   background: #f3f4f6;
207   color: #6b7280;
208   transition: background 0.2s, color 0.2s;
209 }
210 .pill--on { background: #d1fae5; color: #065f46; }
211 .pill--warning { background: #fef3c7; color: #92400e; }
212 .pill--danger { background: #fee2e2; color: #991b1b; }

```

```

213 .pill--extreme { background: #7f1d1d; color: #fff7ed; }
214
215 /* -- Alert log ----- */
216 .alert-card { flex: 1; }
217 .alert-log { max-height: 220px; overflow-y: auto; }
218 .alert-log__empty { font-size: 12px; color: #9ca3af; }
219
220 .alert-row {
221   display: flex;
222   align-items: flex-start;
223   gap: 8px;
224   padding: 8px 0;
225   border-bottom: 1px solid #f3f4f6;
226 }
227 .alert-row:last-child { border-bottom: none; }
228 .alert-row__time { font-size: 11px; color: #9ca3af; white-space: nowrap; min-width: 60px; margin-top: 1px; }
229 .alert-row__msg { font-size: 12px; color: #111827; flex: 1; }
230
231 .alert-pill {
232   font-size: 10px;
233   font-weight: 500;
234   padding: 2px 8px;
235   border-radius: 10px;
236   white-space: nowrap;
237 }
238 .alert-pill--warning { background: #fef3c7; color: #92400e; }
239 .alert-pill--danger { background: #fee2e2; color: #991b1b; }
240 .alert-pill--extreme { background: #7f1d1d; color: #fff7ed; }
241
242 /* -- 24hr history section ----- */
243 .history-section {
244   display: flex;
245   flex-direction: column;
246   gap: 12px;
247 }
248
249 .history-chart-card canvas {
250   width: 100% !important;
251   height: 240px !important;
252 }
253
254 /* -- History table ----- */
255 .history-table-wrap {
256   overflow-x: auto;
257   max-height: 320px;
258   overflow-y: auto;
259 }
260
261 .history-table {
262   width: 100%;
263   border-collapse: collapse;
264   font-size: 12px;
265 }
266
267 .history-table thead th {
268   position: sticky;
269   top: 0;
270   background: #f9fafb;
271   color: #6b7280;
272   font-weight: 500;
273   text-align: left;
274   padding: 8px 10px;
275   border-bottom: 1px solid #e5e7eb;
276   white-space: nowrap;
277   text-transform: uppercase;
278   font-size: 11px;
279   letter-spacing: 0.03em;
280 }
281
282 .history-table tbody tr {
283   border-bottom: 1px solid #f3f4f6;
284   transition: background 0.15s;
285 }
286 .history-table tbody tr:last-child { border-bottom: none; }
287 .history-table tbody tr:hover { background: #f9fafb; }
288 .history-table tbody td {
289   padding: 8px 10px;
290   color: #374151;
291   white-space: nowrap;
292 }
293
294 .hist-badge {
295   display: inline-block;

```

```

296 padding: 2px 9px;
297 border-radius: 10px;
298 font-size: 10px;
299 font-weight: 500;
300 }
301 .hist-badge--safe { background: #d1fae5; color: #065f46; }
302 .hist-badge--warning { background: #fef3c7; color: #92400e; }
303 .hist-badge--danger { background: #fee2e2; color: #991b1b; }
304 .hist-badge--extreme { background: #7f1d1d; color: #fff7ed; }
305
306 .history-empty {
307   font-size: 12px;
308   color: #9ca3af;
309   text-align: center;
310   padding: 1.5rem 0;
311 }
312
313 /* -- Responsive ----- */
314 @media (max-width: 768px) {
315   .topbar {
316     align-items: flex-start;
317     gap: 12px;
318   }
319   .topbar__right {
320     flex-wrap: wrap;
321     justify-content: flex-end;
322   }
323   .metrics { grid-template-columns: repeat(2, 1fr); }
324   .main-grid { grid-template-columns: 1fr; }
325 }
326 @media (max-width: 480px) {
327   .topbar {
328     flex-direction: column;
329   }
330   .topbar__right {
331     justify-content: flex-start;
332     width: 100%;
333   }
334   .metrics { grid-template-columns: 1fr; }
335   .card__header {
336     align-items: flex-start;
337     flex-direction: column;
338   }
339 }

```

F. Dashboard JavaScript

Listing 6. Dashboard Firebase credential template: dashboard/js/secrets.example.js.

```

1  /**
2   * secrets.example.js
3   * Copy this file to secrets.js and replace the placeholder values with the
4   * Firebase web app config from Firebase Project Settings.
5   *
6   * secrets.js is intentionally ignored by Git so local Firebase project details
7   * are not committed or printed in the report source listings.
8   */
9  window.LEAKSENSE_FIREBASE_CONFIG = {
10    apiKey: "YOUR_FIREBASE_WEB_API_KEY",
11    authDomain: "YOUR_PROJECT.firebaseio.com",
12    databaseURL: "https://YOUR_PROJECT-default-rtdb.firebaseio.com",
13    projectId: "YOUR_PROJECT",
14    storageBucket: "YOUR_PROJECT.firebaseio.com",
15    messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
16    appId: "YOUR_FIREBASE_APP_ID",
17  };

```

Listing 7. Firebase JavaScript: dashboard/js/firebase.js.

```

1  /**
2   * firebase.js
3   * Firebase Realtime Database connection.
4   * - leaksense/latest → live reading (every 2s from ESP32)
5   * - leaksense/history → one entry every 5 minutes, kept for 24 hours
6   */
7  const firebaseConfig = window.LEAKSENSE_FIREBASE_CONFIG;
8  const hasFirebaseConfig = firebaseConfig && Object.values(firebaseConfig).every(value =>
9    typeof value === 'string' && value.trim() !== '' && !value.includes('YOUR_')

```

```

10 );
11
12 if (hasFirebaseConfig) {
13   firebase.initializeApp(firebaseConfig);
14 } else {
15   console.warn('Firebase config missing. Copy js/secrets.example.js to js/secrets.js and fill it in.');
```

```

16 }
17
18 const db = hasFirebaseConfig ? firebase.database() : null;
19
20 // -- Helpers -----
```

```

21
22 function numberFrom(...values) {
23   const found = values.find(v => v !== undefined && v !== null && v !== '');
24   const n = Number(found);
25   return Number.isFinite(n) ? n : 0;
26 }
27
```

```

28 function booleanFrom(value) {
29   if (typeof value === 'boolean') return value;
30   if (typeof value === 'string') return value.toLowerCase() === 'true' || value.toLowerCase() === 'on';
31   return Boolean(value);
32 }
33
```

```

34 function normaliseReading(data) {
35   const ppm = numberFrom(data.ppm_compensated, data.ppm, data.gas_ppm, data.gas, data.value);
36   const suppliedVoltage = numberFrom(data.voltage, data.gas_voltage, data.voltage_v, data.sensor_voltage);
37   const voltage = suppliedVoltage > 0 ? suppliedVoltage : gasVoltageFromPpm(ppm);
38   const state = mostSevereState(data.state, getStateFromVoltage(voltage));
39   const alarmActive = state === 'danger' || state === 'extreme';
40
41   return {
42     ppm_compensated: Math.round(ppm),
43     ppm_raw: Math.round(numberFrom(data.ppm_raw, data.raw_ppm, ppm)),
44     voltage,
45     temperature: numberFrom(data.temperature, data.temp),
46     humidity: numberFrom(data.humidity, data.hum),
47     state,
48     thermal_risk: data.thermal_risk === undefined ? booleanFrom(data.thermalRisk) : booleanFrom(data.thermal_risk),
49     fan: data.fan === undefined ? alarmActive : booleanFrom(data.fan),
50     buzzer: data.buzzer === undefined ? alarmActive : booleanFrom(data.buzzer),
51     timestamp: numberFrom(data.timestamp, data.time, Date.now()),
52   };
53 }
54
```

```

55 function snapshotToReadings(snapshot) {
56   const value = snapshot.val();
57   if (!value || typeof value !== 'object' || Array.isArray(value)) return [];
58
59   return Object.values(value)
60     .filter(item => item && typeof item === 'object')
61     .map(normaliseReading)
62     .sort((a, b) => a.timestamp - b.timestamp);
63 }
64
```

```

65 // -- 24-hour history -----
```

```

66
67 const TWENTY_FOUR_HOURS_MS = 24 * 60 * 60 * 1000;
68
69 /**
70 * Loads all history entries from the last 24 hours.
71 * Called once on page load to populate the history chart and table.
72 * @param {function} callback - receives array of normalised reading objects
73 */
```

```

74 function loadHistory(callback) {
75   if (!db) {
76     callback([]);
77     return;
78   }
79
80   const cutoff = Date.now() - TWENTY_FOUR_HOURS_MS;
81
82   db.ref('leaksense/history')
83     .orderByChild('timestamp')
84     .startAt(cutoff)
85     .once('value')
86     .then(snapshot => {
87       const readings = snapshotToReadings(snapshot);
88       callback(readings);
89     })
90     .catch(err => {
91       console.warn('Could not load 24hr history:', err);
92       callback([]);
93     });
94 }
```

```

93     });
94 }
95
96 /**
97 * Listens for new history entries in real time.
98 * Only fires for entries newer than page load time.
99 * @param {function} callback - receives a single normalised reading
100 */
101 function listenToHistory(callback) {
102     if (!db) return false;
103
104     const since = Date.now() - TWENTY_FOUR_HOURS_MS;
105
106     db.ref('leaksense/history')
107         .orderByChild('timestamp')
108         .startAt(since)
109         .on('child_added', snapshot => {
110             const data = snapshot.val();
111             if (!data) return;
112             callback(normaliseReading(data));
113         });
114
115     return true;
116 }
117
118 // -- Live latest reading -----
119
120 /**
121 * Subscribes to leaksense/latest for real-time dashboard updates.
122 * @param {function} callback - onNewReading from dashboard.js
123 */
124 function listenToSensorData(callback) {
125     if (!db) return false;
126
127     db.ref('leaksense/latest').on('value', snapshot => {
128         const data = snapshot.val();
129         if (!data) return;
130         callback(normaliseReading(data));
131     });
132
133     return true;
134 }

```

Listing 8. Main dashboard JavaScript: dashboard/js/dashboard.js.

```

1  /**
2  * dashboard.js
3  * Main dashboard logic.
4  * Handles live readings, alert log, 24hr history table, and state machine.
5  */
6
7  // -- Dashboard display thresholds -----
8  const VOLTAGE_THRESHOLDS = { warning: 1.60, danger: 1.90, extreme: 2.20 };
9  const PPM_THRESHOLDS = { warning: 300, danger: 500, extreme: 800 };
10
11 // -- State -----
12 let alertCount = 0;
13 let prevState = 'safe';
14 const alertLog = [];
15
16 // 24-hour history table data
17 const historyRows = [];
18 const MAX_HISTORY_ROWS = 288; // 24hr at 1 reading per 5 min
19
20 // -- Helpers -----
21
22 function getStateFromPpm(ppm) {
23     return getStateFromVoltage(gasVoltageFromPpm(ppm));
24 }
25
26 function getStateFromVoltage(voltage) {
27     const value = Number(voltage);
28     if (!Number.isFinite(value)) return 'safe';
29     if (value >= VOLTAGE_THRESHOLDS.extreme) return 'extreme';
30     if (value >= VOLTAGE_THRESHOLDS.danger) return 'danger';
31     if (value >= VOLTAGE_THRESHOLDS.warning) return 'warning';
32     return 'safe';
33 }
34
35 const STATE_SEVERITY = { safe: 0, warning: 1, danger: 2, extreme: 3 };
36

```

```

37 function normaliseStateName(state) {
38   const value = String(state || '').trim().toLowerCase();
39   return STATE_SEVERITY[value] === undefined ? 'safe' : value;
40 }
41
42 function mostSevereState(...states) {
43   return states
44     .map(normaliseStateName)
45     .sort((a, b) => STATE_SEVERITY[b] - STATE_SEVERITY[a])[0] || 'safe';
46 }
47
48 function gasVoltageFromPpm(ppm) {
49   const value = Number(ppm);
50   if (!Number.isFinite(value) || value <= 0) return 0;
51
52   if (value < PPM_THRESHOLDS.warning) {
53     return (value / PPM_THRESHOLDS.warning) * VOLTAGE_THRESHOLDS.warning;
54   }
55
56   if (value < PPM_THRESHOLDS.danger) {
57     const span = PPM_THRESHOLDS.danger - PPM_THRESHOLDS.warning;
58     const pct = (value - PPM_THRESHOLDS.warning) / span;
59     return VOLTAGE_THRESHOLDS.warning + pct * (VOLTAGE_THRESHOLDS.danger - VOLTAGE_THRESHOLDS.warning);
60   }
61
62   const span = PPM_THRESHOLDS.extreme - PPM_THRESHOLDS.danger;
63   const pct = Math.min((value - PPM_THRESHOLDS.danger) / span, 1);
64   return VOLTAGE_THRESHOLDS.danger + pct * (VOLTAGE_THRESHOLDS.extreme - VOLTAGE_THRESHOLDS.danger);
65 }
66
67 function formatVoltage(voltage) {
68   const value = Number(voltage);
69   return Number.isFinite(value) ? `${value.toFixed(2)} V` : '-- V';
70 }
71
72 function formatTime(date = new Date()) {
73   return date.toLocaleTimeString('en-AU', {
74     hour: '2-digit', minute: '2-digit', second: '2-digit', hour12: false,
75   });
76 }
77
78 function formatDate(date = new Date()) {
79   return date.toLocaleString('en-AU', {
80     day: '2-digit', month: '2-digit',
81     hour: '2-digit', minute: '2-digit', hour12: false,
82   });
83 }
84
85 function getReadingDate(timestamp) {
86   if (!timestamp) return new Date();
87   const n = Number(timestamp);
88   if (!Number.isFinite(n)) return new Date();
89   return new Date(n < 10000000000 ? n * 1000 : n);
90 }
91
92 // -- UI -- status & metrics -----
93
94 const STATE_MESSAGES = {
95   safe: 'System operating normally. Gas levels within safe range.',
96   warning: 'Warning: elevated gas concentration detected. Monitor the area.',
97   danger: 'DANGER: critical gas level detected. Immediate action required. Fan and buzzer active.',
98   extreme: 'EXTREME: gas voltage is critically high. Evacuate and ventilate immediately.',
99 };
100
101 const VOLTAGE_SUBS = {
102   safe: 'Safe: < 1.60 V',
103   warning: 'Warning: >= 1.60 V',
104   danger: 'Danger: >= 1.90 V',
105   extreme: 'Extreme: >= 2.20 V',
106 };
107
108 const NOTIFICATION_TITLES = {
109   warning: 'LeakSense warning',
110   danger: 'LeakSense danger alert',
111   extreme: 'LeakSense extreme alert',
112 };
113
114 let notificationRegistrationPromise = null;
115
116 function getNotificationRegistration() {
117   if (!('serviceWorker' in navigator)) return Promise.resolve(null);
118
119   if (!notificationRegistrationPromise) {

```

```

120     notificationRegistrationPromise = navigator.serviceWorker
121       .register('service-worker.js')
122       .then(() => navigator.serviceWorker.ready);
123   }
124
125   return notificationRegistrationPromise;
126 }
127
128 function setupAlertNotifications() {
129   const btn = document.getElementById('enableNotificationsBtn');
130   if (!btn || !('Notification' in window)) return;
131
132   function syncButton() {
133     if (Notification.permission === 'granted') {
134       btn.hidden = false;
135       btn.disabled = true;
136       btn.textContent = 'Notifications on';
137       return;
138     }
139
140     btn.hidden = false;
141     btn.disabled = Notification.permission === 'denied';
142     btn.textContent = Notification.permission === 'denied'
143       ? 'Notifications blocked'
144       : 'Enable notifications';
145   }
146
147   btn.addEventListener('click', async () => {
148     try {
149       const permission = await Notification.requestPermission();
150       syncButton();
151
152       if (permission === 'granted') {
153         const registration = await getNotificationRegistration();
154         await showLeakSenseNotification('LeakSense notifications enabled', {
155           body: 'You will be notified when gas reaches warning, danger, or extreme levels.',
156           tag: 'leaksense-notifications-enabled',
157         }, registration);
158       }
159     } catch (err) {
160       console.warn('Could not enable notifications:', err);
161     }
162   });
163
164   getNotificationRegistration().catch(err => {
165     console.warn('Could not register notification service worker:', err);
166   });
167   syncButton();
168 }
169
170 async function showLeakSenseNotification(title, options, registration = null) {
171   const serviceWorkerRegistration = registration || await getNotificationRegistration();
172
173   if (serviceWorkerRegistration && serviceWorkerRegistration.showNotification) {
174     return serviceWorkerRegistration.showNotification(title, options);
175   }
176
177   return new Notification(title, options);
178 }
179
180 function sendAlertNotification(voltage, state, thermalRisk = false) {
181   if (!('Notification' in window) || Notification.permission !== 'granted') return;
182
183   const title = NOTIFICATION_TITLES[state] || 'LeakSense alert';
184   const body = thermalRisk
185     ? `${formatVoltage(voltage)} detected with abnormal temperature rise. ${STATE_MESSAGES.danger}`
186     : `${formatVoltage(voltage)} detected. ${STATE_MESSAGES[state]}`;
187
188   showLeakSenseNotification(title, {
189     body,
190     tag: `leaksense-${state}`,
191     renotify: true,
192     requireInteraction: state === 'danger' || state === 'extreme',
193   }).catch(err => {
194     console.warn('Could not show notification:', err);
195   });
196 }
197
198 function updateStatusBadge(state) {
199   const el = document.getElementById('statusBadge');
200   el.className = `status-badge ${state}`;
201   el.textContent = state.charAt(0).toUpperCase() + state.slice(1);
202 }

```

```

203
204 function updateStateBar(state) {
205     const el = document.getElementById('stateBar');
206     el.className = 'state-bar ${state}';
207     el.textContent = STATE_MESSAGES[state];
208 }
209
210 function updateMetrics(data) {
211     document.getElementById('ppmVal').textContent = formatVoltage(data.voltage);
212     document.getElementById('ppmSub').textContent = data.thermal_risk
213         ? 'Danger: gas + thermal risk'
214         : VOLTAGE_SUBS[data.state];
215     document.getElementById('tempVal').textContent = `${data.temperature}°C`;
216     document.getElementById('humVal').textContent = `${data.humidity}%`;
217     document.getElementById('lastSync').textContent = `Last sync: ${formatTime(getReadingDate(data.timestamp))}`;
218 }
219
220 // -- UI -- device states with yellow LED -----
221
222 function updateDeviceStates(data) {
223     // Fan -- on in Danger/Extreme by default from Firebase normalization
224     const fan = document.getElementById('fanState');
225     fan.textContent = data.fan ? 'on' : 'off';
226     fan.className = 'pill${data.fan ? ' pill--on' : ''}';
227
228     // Buzzer -- on in Danger/Extreme by default from Firebase normalization
229     const buz = document.getElementById('buzzerState');
230     buz.textContent = data.buzzer ? 'on' : 'off';
231     buz.className = 'pill${data.buzzer ? ' pill--on' : ''}';
232
233     // Green LED -- Safe only
234     const green = document.getElementById('greenLed');
235     green.textContent = data.state === 'safe' ? 'on' : 'off';
236     green.className = 'pill${data.state === 'safe' ? ' pill--on' : ''}';
237
238     // Yellow LED -- Warning only
239     const yellow = document.getElementById('yellowLed');
240     yellow.textContent = data.state === 'warning' ? 'on' : 'off';
241     yellow.className = 'pill${data.state === 'warning' ? ' pill--warning' : ''}';
242
243     // Red LED -- Danger and Extreme
244     const red = document.getElementById('redLed');
245     const redActive = data.state === 'danger' || data.state === 'extreme';
246     red.textContent = redActive ? 'on' : 'off';
247     red.className = 'pill${redActive ? ' pill--${data.state}' : ''}';
248 }
249
250 // -- UI -- alert log -----
251
252 function addAlertEntry(voltage, state, thermalRisk = false) {
253     const time = formatTime();
254     const formattedVoltage = formatVoltage(voltage);
255     const labels = {
256         warning: 'Warning threshold crossed',
257         danger: 'DANGER threshold crossed',
258         extreme: 'EXTREME threshold crossed',
259     };
260     const msg = thermalRisk
261         ? `Gas reached ${formattedVoltage} -- thermal risk escalation`
262         : `Gas reached ${formattedVoltage} -- ${labels[state] || 'Alert threshold crossed'}`;
263
264     alertLog.unshift({ time, voltage, state, msg });
265     if (alertLog.length > 20) alertLog.pop();
266
267     alertCount++;
268     document.getElementById('alertCount').textContent = alertCount;
269     renderAlertLog();
270 }
271
272 function renderAlertLog() {
273     const el = document.getElementById('alertLog');
274     if (alertLog.length === 0) {
275         el.innerHTML = '<p class="alert-log__empty">No alerts yet -- system safe.</p>';
276         return;
277     }
278     el.innerHTML = alertLog.map(a => `
279         <div class="alert-row">
280             <span class="alert-row__time">${a.time}</span>
281             <span class="alert-row__msg">${a.msg}</span>
282             <span class="alert-pill alert-pill--${a.state}">${a.state.toUpperCase()}</span>
283         </div>
284     `).join('');
285 }

```

```

286
287 // -- UI -- 24hr history table -----
288
289 /**
290 * Adds a history reading to the table.
291 * Prepends newest rows so most recent is always at the top.
292 * @param {object} reading - normalised reading
293 */
294 function addHistoryRow(reading) {
295   const date = getReadingDate(reading.timestamp);
296   historyRows.unshift({
297     time: formatDateTime(date),
298     voltage: reading.voltage,
299     temp: reading.temperature,
300     hum: reading.humidity,
301     state: reading.state,
302   });
303
304   // Trim to 24hr window
305   if (historyRows.length > MAX_HISTORY_ROWS) historyRows.pop();
306
307   renderHistoryTable();
308 }
309
310 /**
311 * Replaces table with full history array (initial load).
312 * @param {Array} readings - array of normalised readings oldest→newest
313 */
314 function populateHistoryTable(readings) {
315   historyRows.length = 0;
316   // Reverse so newest is first
317   [...readings].reverse().forEach(r => {
318     const date = getReadingDate(r.timestamp);
319     historyRows.push({
320       time: formatDateTime(date),
321       voltage: r.voltage,
322       temp: r.temperature,
323       hum: r.humidity,
324       state: r.state,
325     });
326   });
327   renderHistoryTable();
328 }
329
330 function stateLabel(state) {
331   const map = {
332     safe: '<span class="hist-badge hist-badge--safe">Safe</span>',
333     warning: '<span class="hist-badge hist-badge--warning">Warning</span>',
334     danger: '<span class="hist-badge hist-badge--danger">Danger</span>',
335     extreme: '<span class="hist-badge hist-badge--extreme">Extreme</span>',
336   };
337   return map[state] || state;
338 }
339
340 function renderHistoryTable() {
341   const tbody = document.getElementById('historyTableBody');
342   const empty = document.getElementById('historyEmpty');
343
344   if (historyRows.length === 0) {
345     tbody.innerHTML = '';
346     if (empty) empty.style.display = 'block';
347     return;
348   }
349
350   if (empty) empty.style.display = 'none';
351
352   tbody.innerHTML = historyRows.map(r => `
353     <tr>
354       <td>${r.time}</td>
355       <td><strong>${formatVoltage(r.voltage)}</strong></td>
356       <td>${r.temp.toFixed(1)}°C</td>
357       <td>${r.hum.toFixed(1)}%</td>
358       <td>${stateLabel(r.state)}</td>
359     </tr>
360   `).join('');
361 }
362
363 // -- UI -- live/history view switch -----
364
365 function setupDashboardViews() {
366   const liveView = document.getElementById('liveView');
367   const historyView = document.getElementById('historyView');
368   const showHistoryBtn = document.getElementById('showHistoryBtn');

```

```

369 const showLiveBtn = document.getElementById('showLiveBtn');
370
371 function showView(view) {
372   const showingHistory = view === 'history';
373   liveView.hidden = showingHistory;
374   historyView.hidden = !showingHistory;
375
376   if (showingHistory && historyChart) {
377     historyChart.resize();
378     historyChart.update('none');
379   }
380
381   if (!showingHistory && trendChart) {
382     trendChart.resize();
383     trendChart.update('none');
384   }
385 }
386
387 showHistoryBtn.addEventListener('click', () => showView('history'));
388 showLiveBtn.addEventListener('click', () => showView('live'));
389 showView('live');
390 }
391
392 // -- Main update -- called on every live reading -----
393
394 function onNewReading(data) {
395   const time = formatTime(getReadingDate(data.timestamp));
396   const voltage = Number(data.voltage);
397   if (!Number.isFinite(voltage)) {
398     data.voltage = gasVoltageFromPpm(data.ppm_compensated);
399   }
400   data.state = mostSevereState(data.state, getStateFromVoltage(data.voltage));
401
402   updateStatusBadge(data.state);
403   updateStateBar(data.state);
404   updateMetrics(data);
405   updateDeviceStates(data);
406   updateChart(data.voltage, time); // charts.js
407
408   if (data.state !== 'safe' && data.state !== prevState) {
409     addAlertEntry(data.voltage, data.state, data.thermal_risk);
410     sendAlertNotification(data.voltage, data.state, data.thermal_risk);
411   }
412
413   prevState = data.state;
414 }
415
416 // -- Initialise -----
417
418 document.addEventListener('DOMContentLoaded', () => {
419   initChart(); // live chart -- charts.js
420   initHistoryChart(); // 24hr chart -- charts.js
421   setupDashboardViews();
422   setupAlertNotifications();
423
424   if (typeof listenToSensorData === 'function' && listenToSensorData(onNewReading)) {
425     // Live readings → dashboard
426
427     // 24hr history → chart + table (initial load)
428     loadHistory(readings => {
429       populateHistoryChart(readings); // charts.js
430       populateHistoryTable(readings); // dashboard.js
431     });
432
433     // Stream new history entries in real time
434     listenToHistory(reading => {
435       addHistoryPoint(reading); // charts.js
436       addHistoryRow(reading); // dashboard.js
437     });
438
439     return;
440   }
441
442   // -- Mock fallback -----
443   onNewReading(getMockReading());
444   setInterval(() => {
445     const reading = getMockReading();
446     onNewReading(reading);
447   }, 2000);
448 });

```

Listing 9. Chart JavaScript: dashboard/js/charts.js.

```

1  /**
2  * charts.js
3  * Manages two Chart.js charts:
4  * - trendChart : live readings, last 60 points
5  * - historyChart: 24-hour voltage + temperature + humidity
6  */
7
8  const THRESHOLD_WARNING_V = 1.60;
9  const THRESHOLD_DANGER_V = 1.90;
10 const THRESHOLD_EXTREME_V = 2.20;
11 const GAS_AXIS_MAX_V = 3.00; // max voltage shown on y-axis, for better visual scaling
12 const MAX_LIVE_POINTS = 60;
13
14 let trendChart = null;
15 let historyChart = null;
16
17 // -- Live trend chart -----
18
19 function initChart() {
20   const ctx = document.getElementById('trendChart').getContext('2d');
21
22   trendChart = new Chart(ctx, {
23     type: 'line',
24     data: {
25       labels: [],
26       datasets: [
27         {
28           label: 'Gas (V)',
29           data: [],
30           borderColor: '#3b82f6',
31           backgroundColor: 'rgba(59,130,246,0.07)',
32           borderWidth: 2,
33           pointRadius: 2,
34           pointHoverRadius: 4,
35           tension: 0.4,
36           fill: true,
37         },
38         {
39           label: 'Warning (1.60 V)',
40           data: [],
41           borderColor: 'rgba(217,119,6,0.6)',
42           borderWidth: 1,
43           borderDash: [5,4],
44           pointRadius: 0,
45           fill: false,
46         },
47         {
48           label: 'Danger (1.90 V)',
49           data: [],
50           borderColor: 'rgba(185,28,28,0.6)',
51           borderWidth: 1,
52           borderDash: [5,4],
53           pointRadius: 0,
54           fill: false,
55         },
56         {
57           label: 'Extreme (2.20 V)',
58           data: [],
59           borderColor: 'rgba(127,29,29,0.6)',
60           borderWidth: 1,
61           borderDash: [5,4],
62           pointRadius: 0,
63           fill: false,
64         },
65       ],
66     },
67     options: {
68       responsive: true,
69       maintainAspectRatio: false,
70       animation: { duration: 300 },
71       interaction: { mode: 'index', intersect: false },
72       plugins: {
73         legend: { display: false },
74         tooltip: {
75           callbacks: {
76             label: ctx => ctx.datasetIndex === 0 ? `${Number(ctx.raw).toFixed(2)} V` : null,
77           },
78         },
79       },
80       scales: {
81         x: {
82           display: true,

```

```

83         ticks: { font: { size: 10 }, color: '#9ca3af', maxTicksLimit: 6 },
84         grid: { display: false },
85         border: { display: false },
86     },
87     y: {
88         min: 0,
89         max: GAS_AXIS_MAX_V,
90         ticks: { font: { size: 10 }, color: '#9ca3af', maxTicksLimit: 6 },
91         grid: { color: '#f3f4f6' },
92         border: { display: false },
93     },
94 },
95 },
96 });
97 }
98
99 function updateChart(voltage, time) {
100     const [gasData, warnData, dangerData, extremeData] = trendChart.data.datasets.map(d => d.data);
101     const labels = trendChart.data.labels;
102
103     gasData.push(voltage);
104     warnData.push(THRESHOLD_WARNING_V);
105     dangerData.push(THRESHOLD_DANGER_V);
106     extremeData.push(THRESHOLD_EXTREME_V);
107     labels.push(time);
108
109     if (gasData.length > MAX_LIVE_POINTS) {
110         gasData.shift(); warnData.shift(); dangerData.shift(); extremeData.shift(); labels.shift();
111     }
112
113     trendChart.update('none');
114 }
115
116 function resetChart() {
117     trendChart.data.labels = [];
118     trendChart.data.datasets.forEach(d => { d.data = []; });
119     trendChart.update('none');
120 }
121
122 // -- 24-hour history chart -----
123
124 function initHistoryChart() {
125     const ctx = document.getElementById('historyChart').getContext('2d');
126
127     historyChart = new Chart(ctx, {
128         type: 'line',
129         data: {
130             labels: [],
131             datasets: [
132                 {
133                     label: 'Gas (V)',
134                     data: [],
135                     borderColor: '#3b82f6',
136                     backgroundColor: 'rgba(59,130,246,0.08)',
137                     borderWidth: 2,
138                     pointRadius: 2,
139                     pointHoverRadius: 5,
140                     tension: 0.35,
141                     fill: true,
142                     yAxisID: 'yGas',
143                 },
144                 {
145                     label: 'Temp (°C)',
146                     data: [],
147                     borderColor: '#f59e0b',
148                     backgroundColor: 'transparent',
149                     borderWidth: 1.5,
150                     pointRadius: 0,
151                     tension: 0.35,
152                     fill: false,
153                     yAxisID: 'yEnv',
154                 },
155                 {
156                     label: 'Humidity (%)',
157                     data: [],
158                     borderColor: '#10b981',
159                     backgroundColor: 'transparent',
160                     borderWidth: 1.5,
161                     pointRadius: 0,
162                     tension: 0.35,
163                     fill: false,
164                     yAxisID: 'yEnv',
165                 },
166             ],
167         },
168     });

```

```

166 {
167   label:      'Warning (1.60 V)',
168   data:       [],
169   borderColor: 'rgba(217,119,6,0.5)',
170   borderWidth: 1,
171   borderDash: [5,4],
172   pointRadius: 0,
173   fill:       false,
174   yAxisID:    'yGas',
175 },
176 {
177   label:      'Danger (1.90 V)',
178   data:       [],
179   borderColor: 'rgba(185,28,28,0.5)',
180   borderWidth: 1,
181   borderDash: [5,4],
182   pointRadius: 0,
183   fill:       false,
184   yAxisID:    'yGas',
185 },
186 {
187   label:      'Extreme (2.20 V)',
188   data:       [],
189   borderColor: 'rgba(127,29,29,0.5)',
190   borderWidth: 1,
191   borderDash: [5,4],
192   pointRadius: 0,
193   fill:       false,
194   yAxisID:    'yGas',
195 },
196 ],
197 },
198 options: {
199   responsive:      true,
200   maintainAspectRatio: false,
201   animation:       { duration: 200 },
202   interaction:     { mode: 'index', intersect: false },
203   plugins: {
204     legend: {
205       display: true,
206       position: 'top',
207       labels: { font: { size: 11 }, color: '#6b7280', boxWidth: 14, padding: 12 },
208     },
209     tooltip: {
210       callbacks: {
211         label: ctx => {
212           if (ctx.datasetIndex === 0) return `Gas: ${Number(ctx.raw).toFixed(2)} V`;
213           if (ctx.datasetIndex === 1) return `Temp: ${ctx.raw.toFixed(1)}°C`;
214           if (ctx.datasetIndex === 2) return `Humidity: ${ctx.raw.toFixed(1)}%`;
215           return null;
216         },
217       },
218     },
219   },
220   scales: {
221     x: {
222       display: true,
223       ticks:   { font: { size: 10 }, color: '#9ca3af', maxTicksLimit: 8 },
224       grid:    { display: false },
225       border:  { display: false },
226     },
227     yGas: {
228       type:      'linear',
229       position:  'left',
230       min:       0,
231       max:       GAS_AXIS_MAX_V,
232       title:     { display: true, text: 'Gas (V)', color: '#3b82f6', font: { size: 11 } },
233       ticks:     { font: { size: 10 }, color: '#9ca3af' },
234       grid:      { color: '#f3f4f6' },
235       border:    { display: false },
236     },
237     yEnv: {
238       type:      'linear',
239       position:  'right',
240       min:       0,
241       max:       100,
242       title:     { display: true, text: 'Temp °C / Humidity %', color: '#6b7280', font: { size: 11 } },
243       ticks:     { font: { size: 10 }, color: '#9ca3af' },
244       grid:      { drawOnChartArea: false },
245       border:    { display: false },
246     },
247   },
248 },

```

```

249     });
250 }
251
252 /**
253  * Adds a single history reading to the 24hr chart.
254  * Called both when loading existing history and on new live entries.
255  * @param {object} reading - normalised reading from firebase.js
256  */
257 function addHistoryPoint(reading) {
258     const date = new Date(reading.timestamp);
259     const label = date.toLocaleTimeString('en-AU', { hour: '2-digit', minute: '2-digit', hour12: false });
260
261     const [gasDs, tempDs, humDs, warnDs, dangerDs, extremeDs] = historyChart.data.datasets;
262
263     gasDs.data.push(reading.voltage);
264     tempDs.data.push(reading.temperature);
265     humDs.data.push(reading.humidity);
266     warnDs.data.push(THRESHOLD_WARNING_V);
267     dangerDs.data.push(THRESHOLD_DANGER_V);
268     extremeDs.data.push(THRESHOLD_EXTREME_V);
269     historyChart.data.labels.push(label);
270
271     historyChart.update('none');
272 }
273
274 /**
275  * Replaces all history chart data at once (used on initial load).
276  * @param {Array} readings - array of normalised reading objects
277  */
278 function populateHistoryChart(readings) {
279     historyChart.data.labels = [];
280     historyChart.data.datasets.forEach(d => { d.data = []; });
281
282     readings.forEach(r => addHistoryPoint(r));
283 }

```

Listing 10. Mock data JavaScript: dashboard/js/mock.js.

```

1  /**
2   * mock.js
3   * Simulates ESP32 sensor data for development.
4   * Remove this file (and the script tag in index.html) when Firebase is connected.
5   * Replace with firebase.js instead.
6   */
7
8  // Used only as a local fallback when Firebase is unavailable.
9  let mockBasePpm = 120;
10
11  /**
12   * Adds realistic noise to a base value -- mimics real sensor jitter
13   * @param {number} base
14   * @returns {number}
15   */
16  function addNoise(base) {
17      return Math.max(0, Math.round(base + (Math.random() - 0.5) * 20));
18  }
19
20  /**
21   * Returns a simulated sensor reading matching the exact
22   * same structure the ESP32 will push to Firebase.
23   * Field names must match firebase.js and the ESP32 firmware.
24   *
25   * @returns {{
26   *   ppm_compensated: number,
27   *   ppm_raw: number,
28   *   temperature: number,
29   *   humidity: number,
30   *   state: string,
31   *   fan: boolean,
32   *   buzzer: boolean,
33   *   timestamp: number
34   * }}
35   */
36  function getMockReading() {
37      const ppm = addNoise(mockBasePpm);
38      const state = getStateFromPpm(ppm);
39
40      return {
41          ppm_compensated: ppm,
42          ppm_raw: Math.round(ppm * 1.08), // raw is slightly higher before compensation
43          temperature: parseFloat((22 + Math.random() * 2).toFixed(1)),

```

```

44     humidity:      parseFloat((45 + Math.random() * 5).toFixed(1)),
45     state:         state,
46     thermal_risk:  false,
47     fan:           state !== 'safe',
48     buzzer:        state !== 'safe',
49     timestamp:     Date.now(),
50   };
51 }
52
53 /**
54  * Allows dashboard.js to update the mock base ppm via the slider
55  * @param {number} val
56  */
57 function setMockBasePpm(val) {
58   mockBasePpm = val;
59 }

```

Listing 11. Notification service worker: dashboard/service-worker.js.

```

1 self.addEventListener('notificationclick', event => {
2   event.notification.close();
3
4   event.waitUntil(async () => {
5     const windows = await clients.matchAll({ type: 'window', includeUncontrolled: true });
6     const openWindow = windows.find(client => client.url.includes(self.location.origin));
7
8     if (openWindow) {
9       await openWindow.focus();
10      return;
11    }
12
13    await clients.openWindow('/');
14  })();
15 });

```